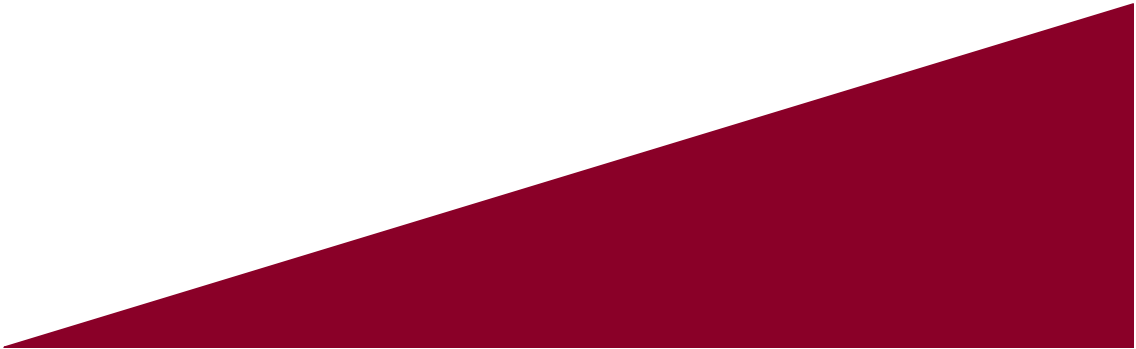




인공지능

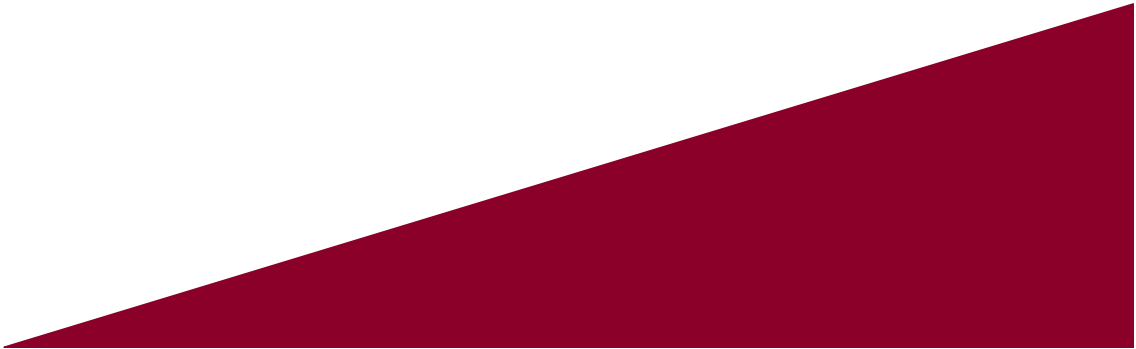
Artificial Intelligence





신경망 기초

본 강의자료는 한빛아카데미 <파이썬으로 만드는 인공지능>을 내용을
기반으로 제작 및 보완되었음을 알립니다.

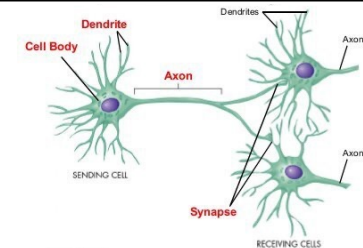
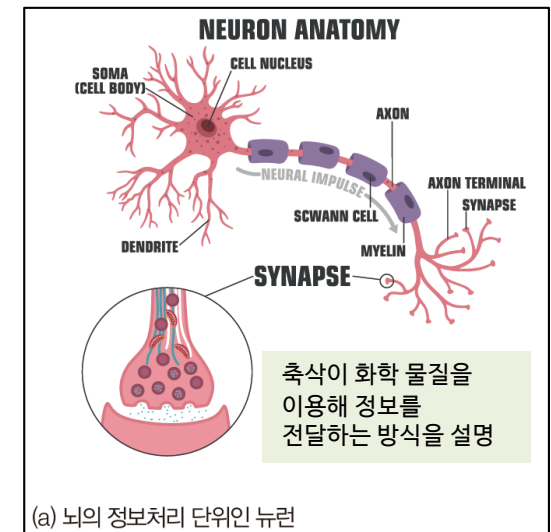


4.1 인공 신경망의 태동

- 인공 신경망은 생물 신경망에서 영감을 얻었지만 실제 구현은 생물 신경망의 원리와 상당히 다름
 - 인공 신경망을 구현하는데 쓸 수 있는 기계는 컴퓨터가 유일한데 컴퓨터의 기본 구조와 원리는 생물의 작동 원리와 근본적으로 다르기 때문임

4.1.1 생물 신경망

- 뉴런은 뇌가 수행하는 정보처리의 단위로서, 세포체^{cell body}, 수상돌기^{dendrite}, 축삭^{axon}으로 구성됨
 - 세포체는 간단한 연산을 수행함
 - 수상돌기는 다른 뉴런으로부터 정보를 받음
 - 축삭은 세포체가 처리한 정보를 다른 뉴런에 전달함
- 시냅스^{synapse}는 한 뉴런에서 다른 뉴런으로 신호를 전달하는 연결 지점
- 뉴런은 서로 연결된 망을 형성하므로 신경망^{neural network}이라 부르는데, 인공 신경망이 등장한 이후에는 서로 구별하기 위해 생물 신경망이라 부름

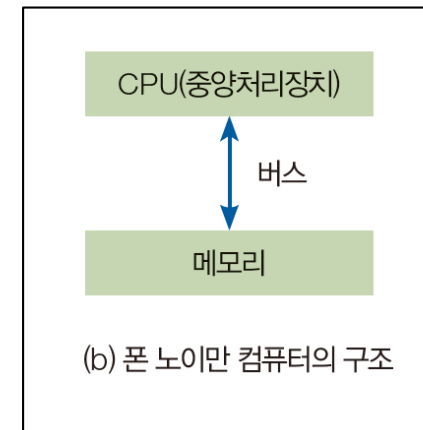


4.1 인공 신경망의 태동

- 인공 신경망은 생물 신경망에서 영감을 얻었지만 실제 구현은 생물 신경망의 원리와 상당히 다름
 - 인공 신경망을 구현하는데 쓸 수 있는 기계는 컴퓨터가 유일한데 컴퓨터의 기본 구조와 원리는 생물의 작동 원리와 근본적으로 다르기 때문임

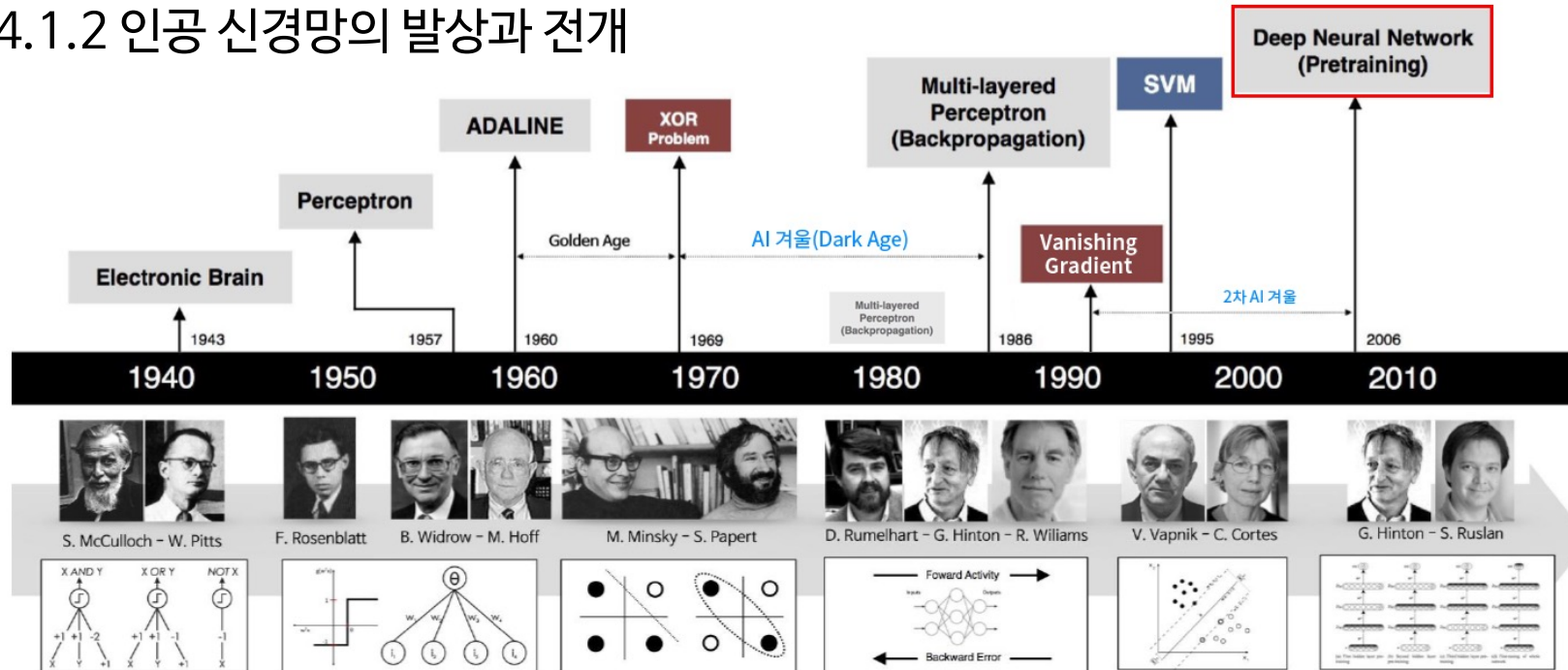
4.1.1 폰 노이만 컴퓨터 Von Neumann computer의 구조

- 현재 우리가 사용하고 있는 컴퓨터는 모두 폰 노이만 구조임
- 메모리는 프로그램(명령어 집합)과 데이터를 저장함
- CPU는 메모리에서 명령어를 한 번에 하나씩 가져가 수행하고 결과를 메모리에 저장함
- 컴퓨터는 아주 빠른 순차 명령어 처리기
- 컴퓨터는 뇌의 병렬 처리와는 근본적으로 다른 정보처리 방식을 사용함



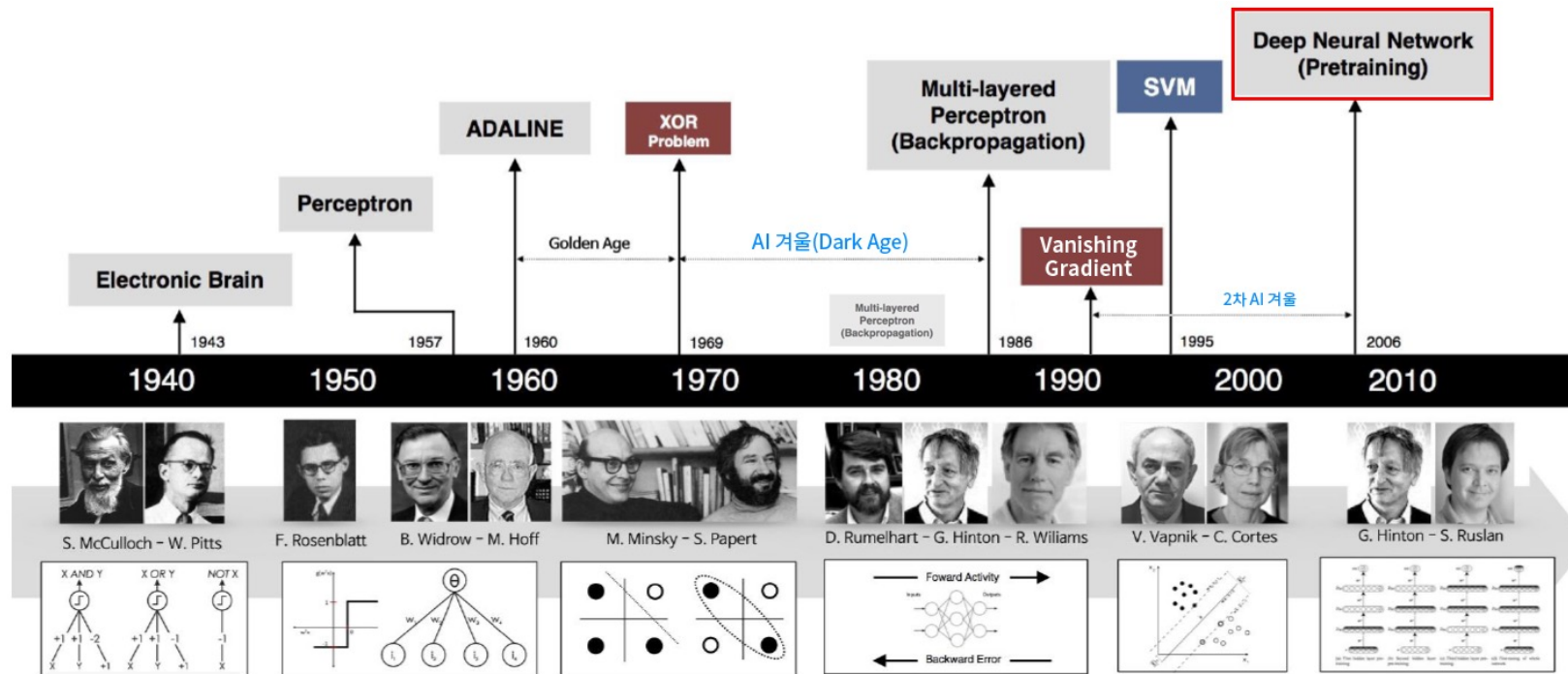
4.1 인공 신경망의 태동

4.1.2 인공 신경망의 발상과 전개



- 1940년대에 생물 신경망 연구자 중 일부는 신경망 동작을 인공적으로 모방하는 일에 관심을 두게 됨
 - 심리학자와 신경의학자가 연구를 주도함
- 1943년에는 맥컬록과 피츠가 신경망의 시초로 볼 수 있는 계산 모형을 발표함
- 1949년에는 헤브가 최초로 학습 알고리즘을 발표함
- 1958년에는 로젠블랫이 **퍼셉트론을 제안함**
- 위드로와 호프는 퍼셉트론의 학습 알고리즘을 개선한 아달린과 아달린을 이어 붙인 마달린을 발표함

4.1 인공 신경망의 태동



- 1960년대에는 신경망으로 모든 문제를 해결할 수 있는 것처럼 과장되어 주목을 받음
- 1969년 민스키와 페퍼트는 **퍼셉트론의 한계를 수학적으로 입증함**
 - 퍼셉트론은 선형 분류기에 불과, XOR 분류도 못한다고 단정함, 이후 신경망에 대한 연구가 크게 퇴조
- 1986년 루멜하트와 동료들이 은닉층을 가진 **다층 퍼셉트론**과 **오류 역전파 알고리즘**을 제시함
 - 이후 신경망 연구는 다시 활기를 찾았고 분류를 포함한 문제해결에 가장 널리 사용되는 도구가 됨
- 1990년대 SVM이 신경망보다 좋은 성능을 보여 신경망이 SVM에 밀리는 형국이 되지만, 딥러닝이 실현되어 다시 신경망이 기계 학습의 주류로 자리 잡게 됨

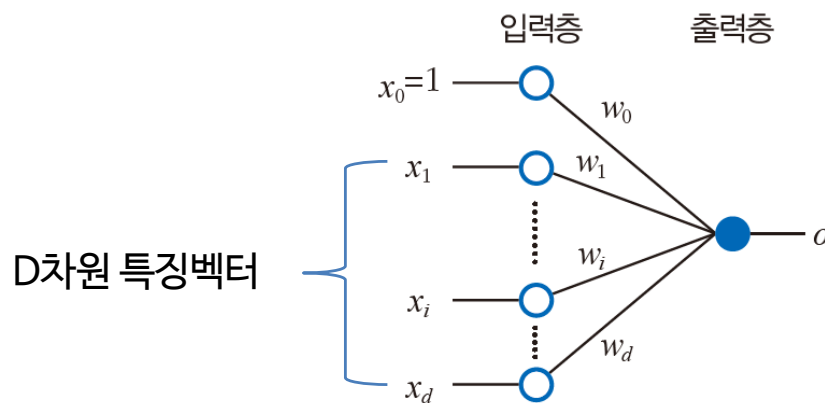
4.2 퍼셉트론의 원리

- 퍼셉트론은 1958년에 탄생 (세월로만 판단하면 아주 낡은 기술)
- 신경망을 공부하려면 퍼셉트론의 원리를 이해하는 것이 중요
 - 왜냐하면 퍼셉트론은 이후에 나온 다층 퍼셉트론과 현재 인공지능 기술을 구현하는데 핵심 역할을 담당하는 딥러닝의 구성요소로 작용하기 때문임
- 퍼셉트론을 제대로 이해하면 현대 인공지능의 중심에 있는 딥러닝을 이해하는데 크게 도움이 됨
- 퍼셉트론은 단순한 모델이어서 기계 학습의 용어와 원리를 설명하는데 적합한데, 이 용어들 원리가 딥러닝에 그대로 쓰이거나 일부 변형되어 적용됨

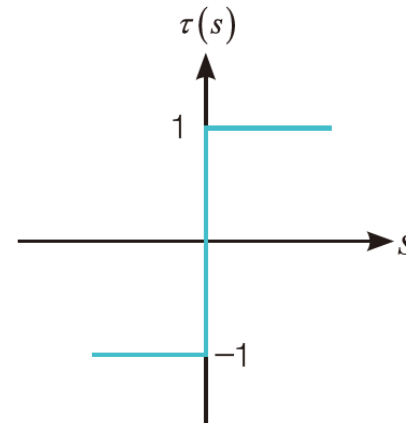
4.2 퍼셉트론의 원리

4.2.1 퍼셉트론의 구조와 연산

- (그림(a)와 같이) 퍼셉트론은 입력층과 출력층으로 구성됨
- 왼쪽에 있는 층은 외부로부터 입력을 받는 입력층(input layer)이고, 오른쪽은 계산 결과를 외부로 내부내는 출력층(output layer)임
- 입력층에는 $d+1$ 개의 노드(node)가 있음. d 는 특징 벡터의 차원인데, iris의 경우 d 는 4 이고, 필기 숫자에서 화소를 특징으로 사용하는 경우 d 는 64임



(a) 퍼셉트론의 구조



(b) 계단 함수를 활성화 함수로 사용

그림 4-2 퍼셉트론의 구조와 동작

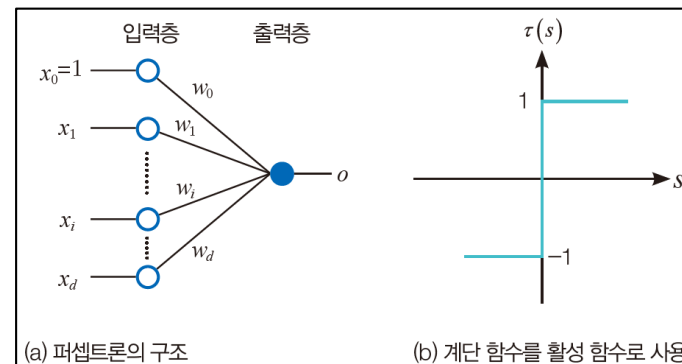
4.2 퍼셉트론의 원리

4.2.1 퍼셉트론의 구조와 연산

- 입력층의 i 번째 노드와 출력층의 노드는 가중치 w_i 를 갖는 에지^{edge}로 연결됨
- i 번째 에지는 특징 x_i 와 가중치 w_i 를 곱해 출력 노드로 전달함
- 0번째 노드의 입력 x_0 은 항상 1인데, 이 노드를 바이어스 노드^{bias node}라 부름
- 출력 노드는 $d+1$ 개의 곱셈 결과를 모두 더한 s 를 계산함
- s 를 **활성 함수** $\tau(s)$ 에 적용한 결과를 출력 o 로 내보냄

$$o = \tau(s) = \tau\left(\sum_{i=1}^d w_i x_i + w_0\right)$$

이때 $\tau(s) = \begin{cases} +1, s > 0 \\ -1, s \leq 0 \end{cases}$ ← 계단 함수



- 활성 함수^{activation function}는 두뇌가 뉴런을 활성화하는 과정을 모방함
- 퍼셉트론은 활성 함수로의 계단 함수^{step function}를 사용함
- 퍼셉트론은 특징 벡터를 위의 식에 따라 1 또는 -1로 변환하는 장치로 볼 수 있음

4.2 퍼셉트론의 원리

4.2.2 “퍼셉트론” 분류 문제 해결하기

- 퍼셉트론을 통해 “이진분류”로 구분하는 인식 문제를 해결할 수 있음
- “이진분류”로 해결 가능한 문제의 예
 - 스팸과 정상메일 구분하는 문제
 - 개/고양이 분류 문제
 - 합격/불합격 예측 문제
 - 생산 라인에서 나오는 제품을 정상/불량으로 구분하는 문제
 - 병원에 온 사람을 환자/건강인으로 구분하는 문제

4.2 퍼셉트론의 원리

4.2.2 퍼셉트론으로 인식하기

[예시 4-1] 퍼셉트론의 인식 능력

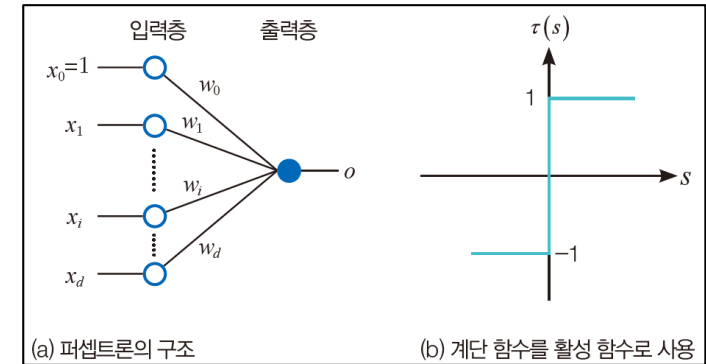
- 어떤 제품이 크기와 색상을 나타내는 2개의 특징으로 표현
 - 특징 벡터는 $X=(\text{크기}, \text{색상})=(x_1, x_2)$ 로 표현할 수 있음
 - 입력 값(특징 값)은 0 또는 1을 갖는다고 가정, 출력 값은 -1 또는 1을 갖는다고 가정
- 생산 라인에서 제품 4개를 수집해 관찰한 결과, 다음과 같이 데이터를 얻고 불량 3개, 정상 1개로 판정

$$x_1=(0,0), \quad x_2=(0,1), \quad x_3=(1,0), \quad x_4=(1,1)$$

$$y_1=-1, \quad y_2=1, \quad y_3=1, \quad y_4=1$$

← 특징 벡터

← 레이블 ($y=1$ 은 불량, $y=-1$ 은 정상)



4.2 퍼셉트론의 원리

4.2.2 퍼셉트론으로 인식하기

[예시 4-1] 퍼셉트론의 인식 능력

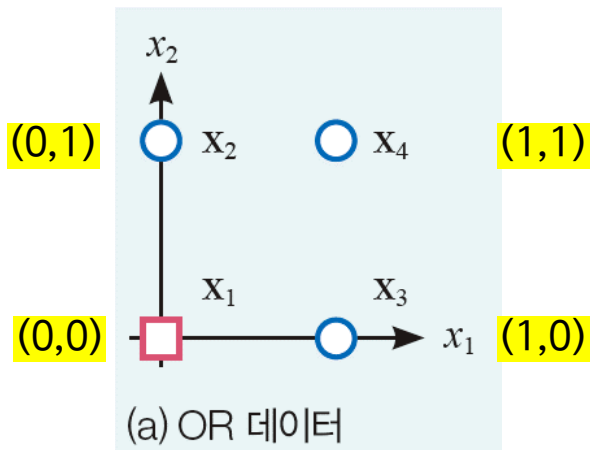
$x_1=(0,0)$, $x_2=(0,1)$, $x_3=(1,0)$, $x_4=(1,1)$

← 특징 벡터

$y_1=-1$, $y_2=1$, $y_3=1$, $y_4=1$

← 레이블 ($y=1$ 은 불량, $y=-1$ 은 정상)

>> 데이터를 그림으로 표현



[NOTE] 논리 연산을 본뜬 OR 데이터

- 왼쪽의 그림 데이터를 분류하는 문제를 OR 분류라고 함
- 1을 참, 0을 거짓으로 간주하면 이진 논리의 OR 연산에 해당되기 때문
- 단순하여 설명하기 쉽기 때문에 초심자에게 신경망의 동작을 설명할 때 종종 사용함

4.2 퍼셉트론의 원리

4.2.2 퍼셉트론으로 인식하기

[예시 4-1] 퍼셉트론의 인식 능력

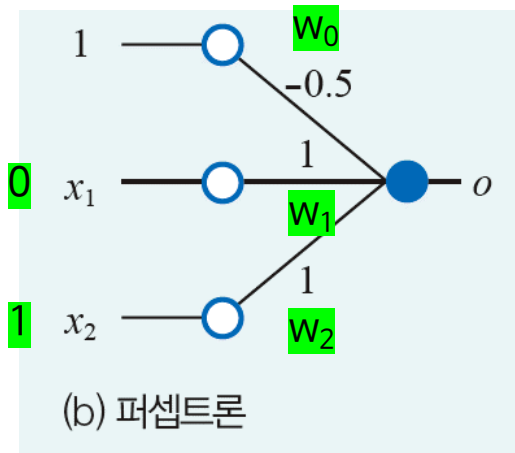
$$\mathbf{x}_1=(0,0), \quad \mathbf{x}_2=(0,1), \quad \mathbf{x}_3=(1,0), \quad \mathbf{x}_4=(1,1)$$

← 특징 벡터

$$y_1=-1, \quad y_2=1, \quad y_3=1, \quad y_4=1$$

← 레이블 ($y=1$ 은 불량, $y=-1$ 은 우량)

>> 샘플 4개를 인식하는 퍼셉트론



$$o = \tau(s) = \tau\left(\sum_{i=1}^d w_i x_i + w_0\right)$$
$$\text{이때 } \tau(s) = \begin{cases} +1, & s > 0 \\ -1, & s \leq 0 \end{cases}$$

→ $x_2=(0,1)$ 샘플을 퍼셉트론에 입력으로 넣으면, 아래와 같은 계산에 의해 예측 값은 1이 나옴 (제대로 인식함, 예측값==참값)

$$\begin{aligned} o &= \tau(w_1 x_1 + w_2 x_2 + w_0) \\ &= \tau(1 * 0 + 1 * 1 - 0.5) = \tau(0.5) = 1 \end{aligned}$$

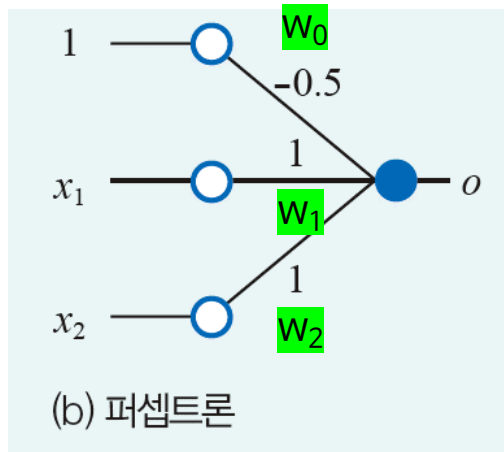
4.2 퍼셉트론의 원리

4.2.2 퍼셉트론으로 인식하기

[예시 4-1] 퍼셉트론의 인식 능력

$x_1=(0,0), x_2=(0,1), x_3=(1,0), x_4=(1,1)$ ← 특징 벡터
 $y_1=-1, y_2=1, y_3=1, y_4=1$ ← 레이블 ($y=1$ 은 불량, $y=-1$ 은 우량)

>> 샘플 4개를 인식하는 퍼셉트론



→ $x_1=(0,0)$ 샘플을 퍼셉트론에 입력으로 넣으면,

$$o = \tau(w_1x_1 + w_2x_2 + w_0) \\ = \tau(1 * 0 + 1 * 0 - 0.5) = \tau(-0.5) = -1$$

→ $x_3=(1,0)$ 샘플을 퍼셉트론에 입력으로 넣으면,

$$o = \tau(w_1x_1 + w_2x_2 + w_0) \\ = \tau(1 * 1 + 1 * 0 - 0.5) = \tau(0.5) = 1$$

→ $x_4=(1,1)$ 샘플을 퍼셉트론에 입력으로 넣으면,

$$o = \tau(w_1x_1 + w_2x_2 + w_0) \\ = \tau(1 * 1 + 1 * 1 - 0.5) = \tau(1.5) = 1$$

모두 맞게 예측

4.2 퍼셉트론의 원리

4.2.2 퍼셉트론으로 인식하기

[예시 4-1] 퍼셉트론의 인식 능력

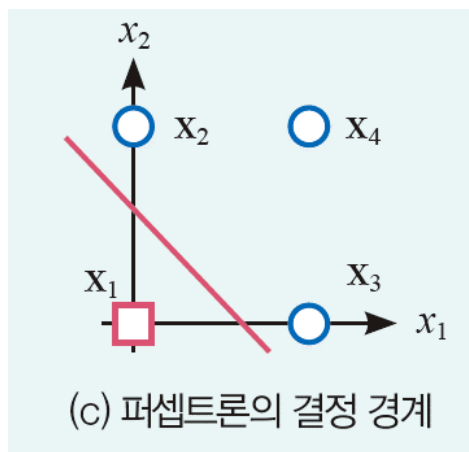
$$x_1=(0,0), \quad x_2=(0,1), \quad x_3=(1,0), \quad x_4=(1,1)$$

← 특징 벡터

$$y_1=-1, \quad y_2=1, \quad y_3=1, \quad y_4=1$$

← 레이블 ($y=1$ 은 불량, $y=-1$ 은 우량)

>> 퍼셉트론 결정 경계



→ $s > 0$ 이면 +1 영역, $s < 0$ 이면 -1 영역, $s = 0$ 이면 결정 경계

$$\begin{aligned} S &= w_1 x_1 + w_2 x_2 + w_0 \\ &= x_1 + x_2 - 0.5 \end{aligned}$$

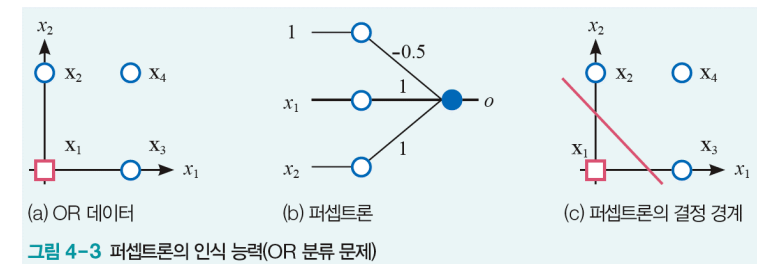
$$x_1 + x_2 - 0.5 = 0 \quad (\text{2개의 부류를 분별하는 결정 직선})$$

$$\therefore x_2 = -x_1 + 0.5$$

특징 공간을 두 공간으로 분할하는 이진 분류기이며, 퍼셉트론의 결정 경계는 선형에 국한됨

- 퍼셉트론의 식을 선형 대수가 사용하는 행렬 표기로 다시 쓰기

4.3 사람의 학습과 신경망의 학습



- [그림4-3]에서는 데이터와 함께 가중치도 함께 주어짐
- 하지만 기계 학습에서는 데이터만 주어지고, 데이터를 제대로 인식하는 퍼셉트론의 가중치를 알아내야 함. 이 과정이 퍼셉트론의 학습과정에 해당함
- (케이스1) [그림4-3]의 OR 데이터에서는 특징 벡터가 2차원이므로 알아내야 하는 가중치는 3개이고 샘플은 4개 뿐이라 최적의 가중치를 쉽게 찾을 수 있음
- (케이스2) iris 데이터의 경우 특징 벡터는 4차원이고 샘플은 150개이므로 사람이 종子和 연필을 들고 가중치 5개를 알아내는 길은 거의 불가능
- (케이스3) 숫자 데이터의 경우는 특징 벡터가 64차원이므로 65개의 가중치를 알아내야 하며, 샘플은 1796개여서 사람이 가중치를 알아내야 일은 불가능
- 따라서, 차원이 높고 샘플의 개수가 아주 많은 상황에서 <퍼셉트론의 가중치를 알아내는 학습 알고리즘>을 구상해야 함

4.3 사람의 학습과 신경망의 학습

4.3.1 사람의 학습 알고리즘

- 기계 학습의 근본 원리는 어떤 면에서는 사람의 학습과 비슷함
- 알고리즘 [4-1]은 사람이 수영을 학습하는 과정을 알고리즘 형식에 맞춰 기술한 것

[알고리즘 4-1] 사람의 수영 학습

```
01. 적절한 동작을 취한다.  
02. while (true)  
03.     동작에 따라 수영을 하고 평가한다.  
04.     if (만족스러움) break          # break문으로 루프 탈출  
05.     더 나은 방향으로 동작을 수정한다.  
06. 동작을 기억한다.
```

- (01행) 처음에는 어떤 동작이 좋은지 모르니 적절하다고 생각되는 동작을 설정하고 수영해 봄
- (03행) 그리고 잘 되는지 평가
- (05행) 만족스럽지 않다면 더 나은 방향으로 동작을 수정한 다음 같은 과정을 반복
- (04행) 이 과정을 반복하다가 만족스러운 상황이 되면 반복 루프를 빠져 나옴
- (06행)이후에 멋진 수영을 하기 위해 찾아낸 동작을 기억함

4.3 사람의 학습과 신경망의 학습

4.3.1 사람의 학습 알고리즘

- 기계 학습의 근본 원리는 어떤 면에서는 사람의 학습과 비슷함
- 알고리즘 [4-1]을 좀 더 수학적으로 기술하면 [알고리즘 4-2]가 됨

[알고리즘 4-2] 사람의 수영 학습(수학적 기술)

```
01. 초기 동작 벡터  $w$ =(팔 돌리는 속도, 팔꿈치 각도, 팔과 귀의 거리)를 초기화한다.  
02. while (true)  
03.      $w$ 에 따라 수영을 하고 동작  $w$ 의 점수  $J(w)$ 를 계산한다.  
04.     if ( $J(w)$ 가 만족스러움) break  
05.     더 나은 방향  $\Delta w$ 를 계산한다.  
06.      $w=w+\Delta w$   
07.  $w$ 를 기억한다.
```

- (01행) 수영을 잘 하기 위해 세 가지 동작의 조합(w)만 생각하겠음
- (01행) 동작을 나타내는 **벡터 w 를 적절한 값으로 초기화**
- (03행) w 에 따라 실제로 수영을 해본 다음 **w 가 얼마나 좋은지 평가, 이때 평가 함수(목적 함수) J 가 있어야 함**
 - 가령 코치가 매기는 점수를 함수 값으로 쓸 수 있음
 - 프로 선수의 경우에는 일정 거리를 가는데 걸리는 시간을 쓸 수도 있음
- (04행) 수영이 만족스럽다면 루프를 탈출하고 학습 종료
- (05행) 그렇지 않으면, **수영을 더 잘 할 수 있는 값을 찾음**
 - 가령 팔을 돌리는 속도를 0.5만큼 줄이고 팔꿈치 각도는 3도만큼 늘리고 팔과 귀의 거리는 1.2만큼 가까이 함. → 이 경우가 $\Delta w = (-0.5, 3, -1.2)$ 임.
- (06행) Δw 를 현재 동작 벡터에 더하여 **다음에 쓸 동작 벡터를 계산**
- (03행) 부터 같은 과정 반복

4.3 사람의 학습과 신경망의 학습

4.3.2 신경망의 학습 알고리즘

- 신경망의 학습은 사람의 학습과 비슷한 점도 있고 다른 점도 있음
 - 조금씩 나은 방향으로 찾아 개선해 나가는 절차는 같음
 - 철저히 수학에 의존한다는 점에서 사람의 학습과 크게 다름

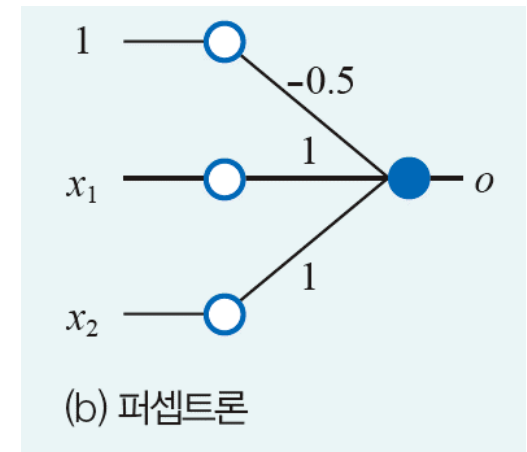
[알고리즘 4-3] 신경망의 학습

입력: 훈련 데이터

출력: 최적의 매개변수 값

01. 난수로 매개변수 벡터 $\mathbf{w}$ 를 초기화한다. # $\mathbf{w}$ 는 신경망의 가중치([그림 4-2(a)]의 $(w_0, w_1, \dots, w_d)$)
02. while (true)
03. $\mathbf{w}$ 에 따라 데이터를 인식하고 손실 함수 $J(\mathbf{w})$ 를 계산한다.
04. if ($J(\mathbf{w})$ 가 만족스러움) break
05. 손실 함수 값을 낮추는 방향 $\Delta\mathbf{w}$ 를 계산한다.
06. $\mathbf{w} = \mathbf{w} + \Delta\mathbf{w}$
07. $\mathbf{w}$ 를 저장한다.

- (01행) 난수를 생성해 매개변수를 초기화
 - 처음에는 매개변수 값을 어떻게 설정해야 좋을 지 알지 못하므로 난수 사용
 - 물론 난수로 설정했기 때문에 성능이 형편없는 신경망 일 것임
- (03행) 손실 함수 J 를 이용해 현재 매개변수 값 $\mathbf{w}$ 의 성능을 측정함.
 - 손실 함수는 여러 행태가 있는데 기본적으로 신경망이 범하는 오류와 비례 관계임
- (04행) 현재 매개변수 값의 성능, 즉 $J(\mathbf{w})$ 가 충분히 낮으면 루프를 탈출, 그렇지 않으면
- (05행)으로 가서 손실 함수 값이 낮아지는, 즉 신경망이 오류를 덜 범하는 방향을 탐색
- (06행) 탐색을 반영하여 매개변수 값을 갱신
- (07행) 나중에 사용할 목적으로 매개변수 값을 저장



4.3 사람의 학습과 신경망의 학습

4.3.2 신경망의 학습 알고리즘

- 신경망을 학습하는 일은 전형적인 최적화 문제optimization problem임
 - [알고리즘 4-3]은 전형적인 최적화 알고리즘임
- 최적화 알고리즘은 매개변수에 대해 최적의 값을 알아내야 하는데, [알고리즘4-3]의 경우 매개변수는 신경망의 가중치임
 - 퍼셉트론의 경우 $w_0, w_1, \dots, w_d$ 가 매개변수임

[알고리즘 4-3] 신경망의 학습

입력: 훈련 데이터

출력: 최적의 매개변수 값

01. 난수로 매개변수 벡터 $\mathbf{w}$ 를 초기화한다. # $\mathbf{w}$ 는 신경망의 가중치([그림 4-2(a)]의 $(w_0, w_1, \dots, w_d)$)
02. while (true)
03. $\mathbf{w}$ 에 따라 데이터를 인식하고 손실 함수 $J(\mathbf{w})$ 를 계산한다.
04. if ($J(\mathbf{w})$ 가 만족스러움) break
05. 손실 함수 값을 낮추는 방향 $\Delta\mathbf{w}$ 를 계산한다.
06. $\mathbf{w} = \mathbf{w} + \Delta\mathbf{w}$
07. $\mathbf{w}$ 를 저장한다.

4.3 사람의 학습과 신경망의 학습

4.3.2 신경망의 학습 알고리즘

- [알고리즘 4-3]을 구현하려면 몇 가지를 구체화 해야함
 - (01행) 매개변수 초기화 방법 **0초기화는 피하라**
 - (03행) 손실 함수 정의
 - (04행) 학습 종료를 위한 조건
 - (05행) 손실 함수의 값을 낮추는, **즉 신경망의 오류를 낮추는 방향을 찾는 것****
 - 신경망은 손실 함수의 값을 낮추는 방향을 찾기 위해 미분을 사용

[알고리즘 4-3] 신경망의 학습

입력: 훈련 데이터

출력: 최적의 매개변수 값

01. 난수로 매개변수 벡터 $\mathbf{w}$ 를 초기화한다. # $\mathbf{w}$ 는 신경망의 가중치([그림 4-2(a)]의 $(w_0, w_1, \dots, w_d)$)
02. while (true)
03. $\mathbf{w}$ 에 따라 데이터를 인식하고 손실 함수 $J(\mathbf{w})$ 를 계산한다.
04. if ($J(\mathbf{w})$ 가 만족스러움) break
05. 손실 함수 값을 낮추는 방향 $\Delta\mathbf{w}$ 를 계산한다.
06. $\mathbf{w} = \mathbf{w} + \Delta\mathbf{w}$
07. $\mathbf{w}$ 를 저장한다.

4.4 퍼셉트론 학습 알고리즘

- 퍼셉트론을 학습하려면 **손실 함수** $\text{loss function } J$ 를 잘 설계해야 하며, **손실 함수의 값을 낮추는 방향을 찾는 방법** $\text{optimization function}$ 을 고안해야 함

4.4.1 손실 함수

- 손실 함수 loss function 는 오차함수 error function , 비용 함수 cost function 라고 부름
- (질문) 신경망의 예측 결과를 평가하는 방법 즉, 예측 결과가 정답과 얼마나 가까운지 알 수 있는 방법은?
- (정답) 오차 함수를 통해 오차를 측정하는 것

4.4 퍼셉트론 학습 알고리즘

- 오차 함수란
 - 신경망의 예측 결과가 정답과 비교해서 얼마나 ‘동떨어졌는지’ 측정하는 수단
 - 손실 값이 크면 AI모델의 정확도가 낮다는 의미
 - 손실 값이 작으면 AI모델의 정확도가 높다는 의미
 - 손실이 클수록 정확도를 개선하기 위해 모델을 더 많이 학습해야 함
- 오차 함수가 왜 필요한가
 - 최적화 문제는 오차 함수를 정의하고 매개변수를 조정하면서 오차 함수의 오차를 최소가 되도록 하는 문제에 해당함
 - 최적화 문제의 목표는 오차 함수 값이 최소가 되게 하는 최적의 파라미터를 찾는 것
 - 이와 같이 오차를 최소로 만드는 과정을 오차 함수 최적화(error function optimization)라 함

4.4 퍼셉트론 학습 알고리즘

- 오차의 값은 언제나 양수
 - 예측 오차는 양수여야 평균을 계산할 때 오차끼리 서로 상쇄되는 일이 발생하지 않음
- 오차끼리 상쇄되는 예시
 - (질문) 신경망으로 예측하려는 2개의 데이터 점이 있고, 첫 번째 점에 대한 예측 오차가 '10' 이고, 두 번째 점에 대한 예측의 오차가 '-10' 이라면, 평균 오차는?
 - (정답) '0' 임
 - (문제점) 오차가 '0'이면 신경망이 완벽하게 정답을 맞췄다는 의미로 이해하고 파라미터(가중치) 업데이트가 발생하지 않음
 - (해결책) 이런 문제를 방지하고자 모든 오차는 <양수>로 계산되어 평균 오차를 계산해야 함

4.4 퍼셉트론 학습 알고리즘

- 오차 함수의 예 : 평균제곱오차

- **평균제곱오차** mean squared error, MSE 는 출력 값이 실수인 회귀 문제에서 널리 사용되는 오차 함수
- 평균제곱오차는 각 데이터 점의 오차를 제곱해서 평균을 구함
- 오차를 제곱하기 때문에 오차가 항상 양의 값을 가지며, 오차 값이 그대로 결과에 대한 평가가 되므로 계산이 간편함
- 평균제곱오차(MSE)는 오차를 제곱하기 때문에 예외 값에 민감함
- 문제의 종류에 따라 이런 민감함은 단점 혹은 장점으로 작용함
- **절대제곱오차** mean absolute error, MAE 는 MSE의 변종으로 오차의 절대값의 평균을 계산함

$$\text{MSE} \rightarrow E(W, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

$$\text{MAE} \rightarrow E(W, b) = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

$E(W, b)$: 오차 함수

W : 가중치 행렬

b : 편향 벡터

N : 학습 데이터 수

$\hat{y}_i$: 출력된 예측 결과

y_i : 실제 정답(레이블값)

$(\hat{y}_i - y_i)$: 오차 또는 잔차라고 부름

4.4 퍼셉트론 학습 알고리즘

- 오차 함수의 예 : 교차 엔트로피
 - 교차 엔트로피(cross-entropy) 는 두 확률 분포 간의 차이를 측정할 수 있다는 특성 덕분에 분류 문제에서 많이 사용함 (유사할 수록 0에 가까워짐)
 - 즉, 샘플의 정답 분포와 예측 분포의 어긋남에 주는 벌점

y가 대상확률 분포, p가 예측 확률 분포, m이 클래스 수일 경우,

학습 샘플 (n=1)에 대한 교차 엔트로피는

$$E(W, b) = -\sum_{i=1}^m \hat{y}_i \log(p_i)$$

모든 학습 샘플 n에 대한 교차 엔트로피는

$$E(W, b) = -\sum_{i=1}^n \sum_{j=1}^m \hat{y}_{ij} \log(p_{ij})$$

4.4 퍼셉트론 학습 알고리즘

$$E(W, b) = -\sum_{i=1}^m \hat{y}_i \log(p_i)$$

- 오차 함수의 예 : 교차 엔트로피
 - 예시)

	(고양이) 확률	(개) 확률	(물고기) 확률	교차엔트로피
정답	0.0	1.0	0.0	
예측 #1	0.2	0.3	0.5	1.2
예측 #2	0.3	0.5	0.2	0.69
⋮				
예측#n	0	1	0	0

모델 학습 진행

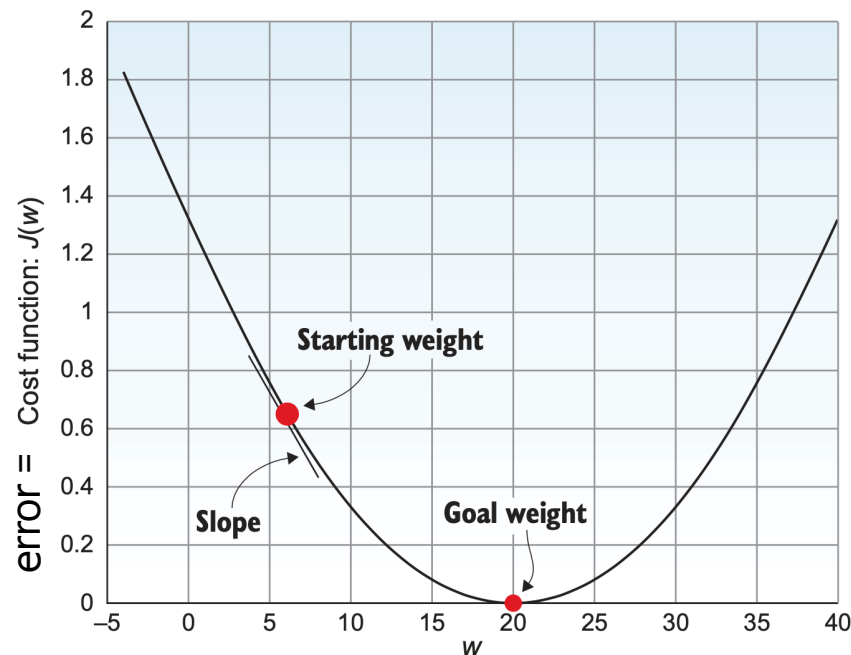
손실이 줄어듦

$$E_1 = - (0.0 * \log(0.2) + 1.0 * \log(0.3) + 0.0 * \log(0.5)) = 1.2$$

$$E_2 = - (0.0 * \log(0.3) + 1.0 * \log(0.5) + 0.0 * \log(0.2)) = 0.69$$

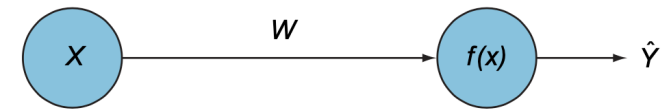
4.4 퍼셉트론 학습 알고리즘

- 오차 함수와 가중치의 관계
 - 가중치는 오차를 줄이기 위해 조절하는 신경망 파라미터
 - 신경망은 가중치를 업데이트하는 방식으로 학습을 진행



- ✓ 모델 학습의 목표는 w 가 오차함수의 하강(descend)을 따라 내려가 오차가 최소가 되는 목표 지점에 도달하는 것
- ✓ 목표 지점을 찾아가는 과정은 <최적화 알고리즘>을 이용해서 반복적으로 가중치를 갱신하는 것

입력이 하나인 퍼셉트론



$$\begin{aligned}\text{error} &= |\hat{y} - y| \\ &= |(w \cdot x) - y|\end{aligned}$$

$x=0.3, y=0.8$ 데이터가 주어지면

$$\hat{y}_i = w \cdot x = w \cdot 0.3$$

$$\begin{aligned}\text{error} &= |\hat{y} - y| \\ &= |w \cdot 0.3 - 0.8|\end{aligned}$$

w 는 무작위 값으로 초기화 되어 있고,
error가 작아지는 방향으로 w 값을 업데이트 해야함

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

- 최적화 알고리즘

- 신경망 학습과정은 여러 개의 표본(학습 데이터)을 신경망에 입력해 순방향 계산을 통해 예측 결과를 계산한 다음 그 표본의 정답과 비교해서 오차를 계산하고 마지막으로 오차값이 최소가 될 때까지 **신경망의 가중치를 조정(업데이트) 하는 것**

- 최적화란

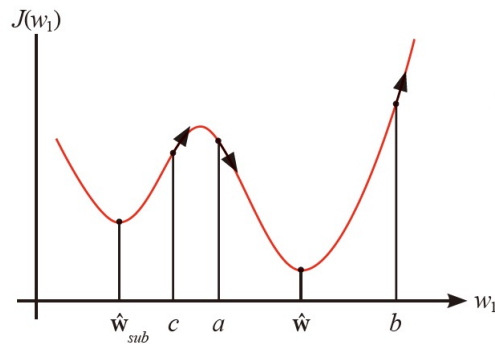
↓
최적의 가중치를 찾아줄 알고리즘은?

- 최적화의 정의
 - 어떤 함수 값을 최소화 하거나 최대화 하는 것으로 문제를 바라보는 관점
 - 신경망 학습에 오차함수를 도입함으로써 오차를 최소화 하는 최적화 문제로 해결
- 최적화 알고리즘의 필요성
 - 가중치의 모든 가능한 값을 계산해 보고 오차의 최소 지점을 찾는 방법은 이론적으로 가능하지만 경우의 수가 너무 많아 현실적으로 불가능함
- 신경망에서 사용되는 최적화 알고리즘
 - 경사 하강법

4.4 퍼셉트론 학습 알고리즘

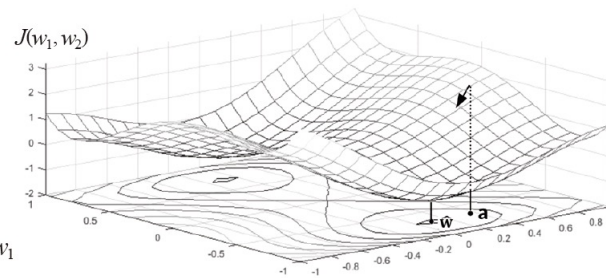
4.4.2 학습 알고리즘 설계

- 경사 하강법 gradient descent의 원리
 - 편의상 매개변수가 한 개인 경우와 두 개인 경우로 나누어 설명
 - 학습 알고리즘 최적화 함수는 손실 함수 오차 함수 J 의 최저점 $\hat{\mathbf{w}}$ 을 찾아야 함



(a) 매개변수가 1개인 경우: $\mathbf{w}=(w_1)$

그림 4-5 경사 하강법



(b) 매개변수가 2개인 경우: $\mathbf{w}=(w_1, w_2)$

- 손실 함수 예시 가정

$$J(\mathbf{w}) = \sum_{\mathbf{x} \in I} -y(\mathbf{w}\mathbf{x}^T) \quad (4.5)$$

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

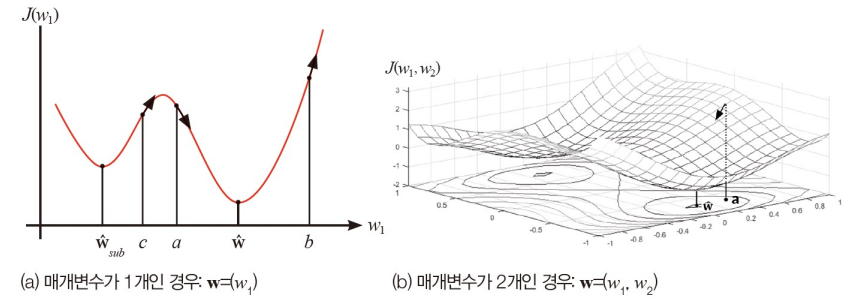


그림 4-5 경사 하강법

■ 매개 변수가 하나인 경우

- **경사 하강법은 미분을 이용하여 최적해를 찾는 기법**
- 미분값 $\partial J / (\partial w_1)$ 의 반대 방향이 최적해에 접근하는 방향이므로 현재 w_1 에 $-\partial J / (\partial w_1)$ 를 더하면 최적해에 가까워짐 → 식 (4.6)의 학습 규칙
- 방향은 알지만 얼마나 가야하는지에 대한 정보가 없기 때문에 학습률 ρ 를 곱하여 조금씩 이동(ρ 는 하이퍼 매개변수로서 보통 0.001이나 0.0001처럼 작은 값 사용)

$$w_1 = w_1 + \rho \left(-\frac{\partial J}{\partial w_1} \right) \quad (4.6)$$

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

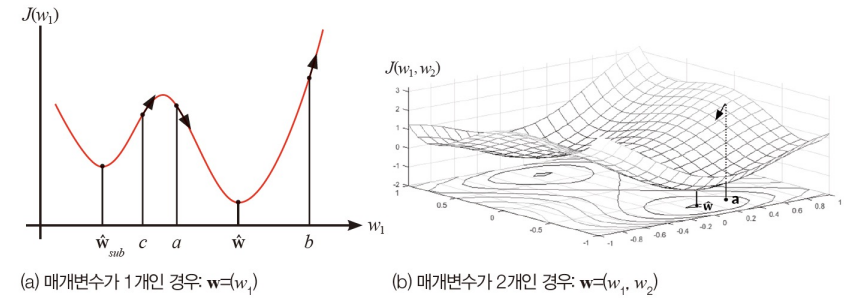


그림 4-5 경사 하강법

■ 매개 변수가 여럿인 경우

- 편미분으로 구한 그래디언트를 사용 (매개변수 별로 독립적으로 미분)

$$\left. \begin{aligned} \mathbf{w} &= \mathbf{w} + \rho(-\nabla \mathbf{w}) \\ \nabla \mathbf{w} &= \left(\frac{\partial J}{\partial w_0}, \frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \dots, \frac{\partial J}{\partial w_d} \right) \end{aligned} \right\} \quad (4.7)$$

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

- 매개 변수가 여럿인 경우

- 퍼셉트론에 식(4.7) 적용
- 식 (4.5)를 매개변수 w_i 로 편미분하면,

$$\frac{\partial J}{\partial w_i} = \sum_{\mathbf{x} \in I} \frac{\partial (-y(w_0 + w_1 x_1 + \dots + w_i x_i + \dots + w_d x_d))}{\partial w_i} = \sum_{\mathbf{x} \in I} -y x_i$$

- 정리하면,

$$\frac{\partial J}{\partial w_i} = \sum_{\mathbf{x} \in I} -y x_i, \quad i = 0, 1, 2, \dots, d \quad (4.8)$$

- 식 (4.8)을 식 (4.7)에 대입하면.

$$\text{퍼셉트론 학습 규칙: } w_i = w_i + \rho \sum_{\mathbf{x} \in I} y x_i, \quad i = 0, 1, 2, \dots, d \quad (4.9)$$

- 식 (4.9)를 행렬 형태로 쓰면, 퍼셉트론 학습 알고리즘이 사용하는 핵심 공식

$$\text{퍼셉트론의 학습 규칙(행렬 표기): } \mathbf{w} = \mathbf{w} + \rho \sum_{\mathbf{x} \in I} y \mathbf{x} \quad (4.10)$$

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

- 퍼셉트론의 학습 알고리즘

- [알고리즘 4-4]: 식 (4.10)을 [알고리즘 4-3]에 적용

- 03~08행을 수행하여 훈련 집합에 있는 샘플 전체를 한번 처리하는 일을 세대 epoch라 부름

[알고리즘 4-4] 퍼셉트론의 학습

입력: 훈련 집합(n 은 샘플의 개수)

출력: 최적의 매개변수 $\hat{\mathbf{w}}$

```
01. 난수로 매개변수 벡터  $\mathbf{w}$ 를 초기화한다. #  $\mathbf{w}$ 는 퍼셉트론 가중치
02. while (true)
03.    $I = []$                                 # 공집합
04.   for  $j=1$  to  $n$ 
05.      $o = \tau(\mathbf{w}\mathbf{x}_j^T)$                   # 식 (4.4)를 적용해 인식 수행
06.     if( $o \neq y_j$ )  $I = I \cup \mathbf{x}_j$         # 틀린 샘플을  $I$ 에 추가
07.     if( $I = \text{공집합}$ ) break                # 모든 샘플을 맞히면(손실 함수가 0) 루프를 빠져 나감
08.      $\mathbf{w} = \mathbf{w} + \rho \sum_{\mathbf{x} \in I} y\mathbf{x}$       # 식 (4.10)을 적용해 매개변수 갱신
09.    $\hat{\mathbf{w}} = \mathbf{w}$ 
```

- [알고리즘 4-4]는 데이터가 선형 분리 불가능한 경우 무한 루프

- 여러 세대에 걸쳐 I 의 크기가 줄지 않으면 수렴했다고 판단하고 루프를 빠져나오도록 수정 필요

4.4 퍼셉트론 학습 알고리즘

4.4.2 학습 알고리즘 설계

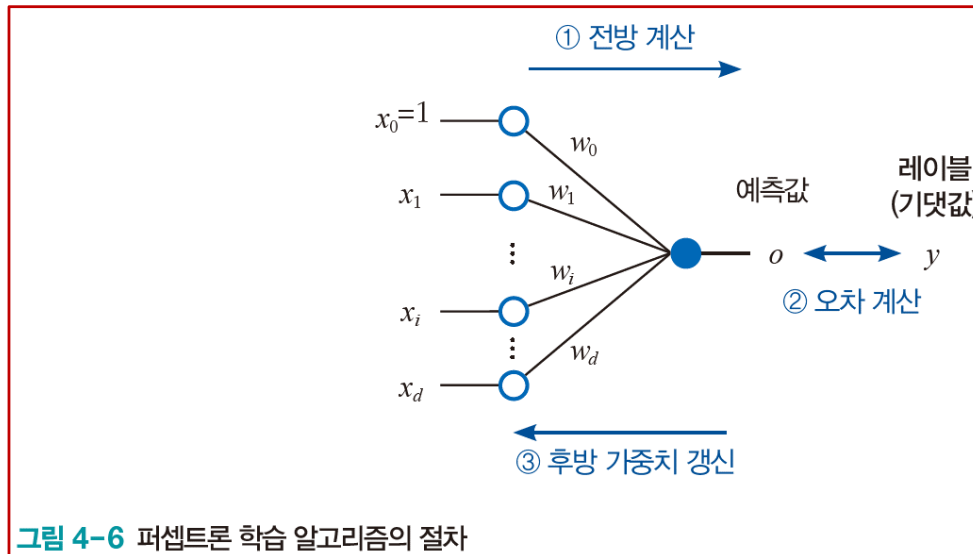
- 퍼셉트론의 학습 알고리즘을 개념적으로 설명
 - 알고리즘은 전방 forward 계산(05행)
 - 오차 계산(06행)
 - 후방 가중치(다음 가중치) 갱신을 반복(08행)
 - 딥러닝을 포함하여 신경망 학습 알고리즘은 모두 이 절차를 따름
 - 딥러닝은 많은 층을 거쳐 전방 계산과 후방 가중치 갱신을 수행

[알고리즘 4-4] 퍼셉트론의 학습

입력: 훈련 집합(n 은 샘플의 개수)

출력: 최적의 매개변수 $\hat{\mathbf{w}}$

```
01. 난수로 매개변수 벡터  $\mathbf{w}$ 를 초기화한다. #  $\mathbf{w}$ 는 퍼셉트론 가중치
02. while (true)
03.    $I = []$  # 공집합
04.   for  $j=1$  to  $n$ 
05.      $o = \tau(\mathbf{w}\mathbf{x}_j^T)$  # 식 (4.4)를 적용해 인식 수행
06.     if( $o \neq y_j$ )  $I = I \cup \mathbf{x}_j$  # 틀린 샘플을  $I$ 에 추가
07.     if( $I = \text{공집합}$ ) break # 모든 샘플을 맞히면(손실 함수가 0) 루프를 빠져 나감
08.      $\mathbf{w} = \mathbf{w} + \rho \sum_{\mathbf{x} \in I} y\mathbf{x}$  # 식 (4.10)을 적용해 매개변수 갱신
09.  $\hat{\mathbf{w}} = \mathbf{w}$ 
```



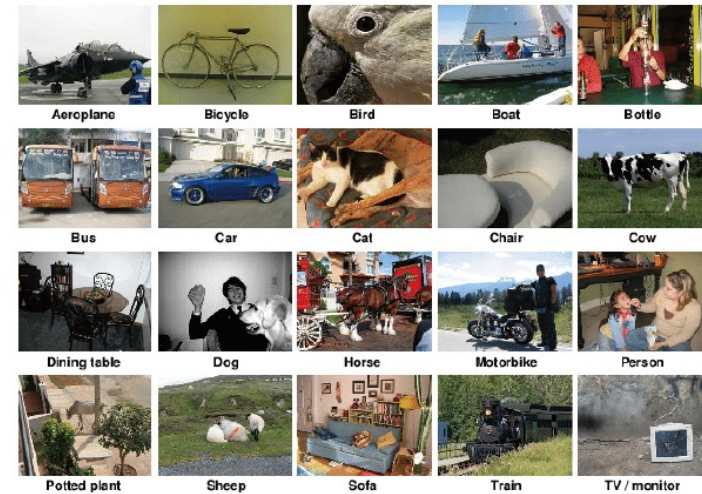
4.5 전통 기계 학습 → 현대 기계 학습으로 확장

- 현대 기계 학습에서 가장 강력한 모델인 딥러닝도 퍼셉트론이 복잡해진 것
- 기본 원리는 퍼셉트론과 딥러닝 모두 동일함

4.5 현대 기계 학습으로 확장

4.5.1 현대적인 기계 학습의 복잡도

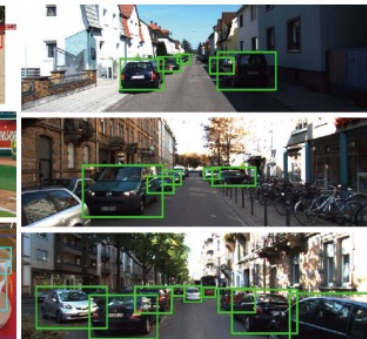
- 최근 관심 많은 문제
 - 분류 classification
지정된 몇가지 부류로 구분하는 문제
 - 물체 검출 object detection
: 물체 위치를 바운딩 박스로 찾아내는 문제
 - 물체 분할 object segmentation
: 화소 수준으로 물체를 찾아내는 문제
(추적 대상 물체를 형광색으로 칠하는 응용)



(a) 분류 문제



(b) 검출 문제



(c) 분할 문제

그림 4-7 세 종류의 문제와 데이터셋

4.5 현대 기계 학습으로 확장

4.5.1 현대적인 기계 학습의 복잡도

- 적절한 손실 함수 필요
 - 분류 : 참 값과 예측 값의 차이를 손실 함수에 반영
 - 물체 검출 : 참 바운딩 박스와 예측한 바운딩 박스의 겹침 정도를 손실 함수에 반영
 - 물체 분할 : 화소(픽셀) 별로 레이블이 지정한 물체에 해당하는지 따지는 손실 함수
- 실용적인 신경망의 방대한 매개변수 개수
 - 예) 딥러닝 모델 ResNet50은 50개의 층을 가지며 매개변수(W , 가중치)는 2천300만개 이상
 - 다행히 1~2차원에서 개발한 공식이 고차원(수십~수천만)에 그대로 적용
 - 매개변수가 많아지면 과잉 적합 overfitting 가능성이 커지므로, 과잉 적합을 방지하기 위해 더 많은 데이터 사용 또는 더 정교한 규제 regularization 기법 적용 필요
 - 계산 시간이 많이 소요되므로 병렬 처리를 위해 GPU 필요

4.5 현대 기계 학습으로 확장

4.5.2 스토케스틱 경사 하강법 stochastic gradient descent

- 경사 하강법을 여러 방식으로 개선하여 사용함 (배치, 패턴, 미니배치 등)
- 패턴 모드와 미니배치 모드의 경사 하강법에서는 **랜덤 샘플링을 적용**하기 때문에 스토케스틱이라는 수식어를 붙여, 스토케스틱 경사 하강법 (SGD, stochastic gradient descent) 이라 부름
- 배치 모드 batch mode
 - 한 세대에 매개변수 갱신이 단 한번만 일어나는 방식
- 패턴 모드 pattern mode
 - 한 세대에 매개변수 갱신이 샘플 개수만큼 일어나는 방식
 - 새로운 세대를 시작할 때 샘플을 뒤섞어 세대마다 처리 순서를 다르게 함
 - 순서를 뒤섞으면 **랜덤 샘플링** 효과가 발생
- 미니배치 모드 mini-batch mode
 - 배치 방식과 패턴 방식의 중간을 사용하는 셈
 - 훈련 집합을 일정한 크기의 부분 집합으로 나눈 다음 부분 집합 별로 처리
 - 부분 집합으로 나눌 때 **랜덤 샘플링**이 적용

4.6 퍼셉트론 프로그래밍

4.6.1 OR 데이터 인식

- 》 실습 4-1. OR 데이터 퍼셉트론으로 분류하기

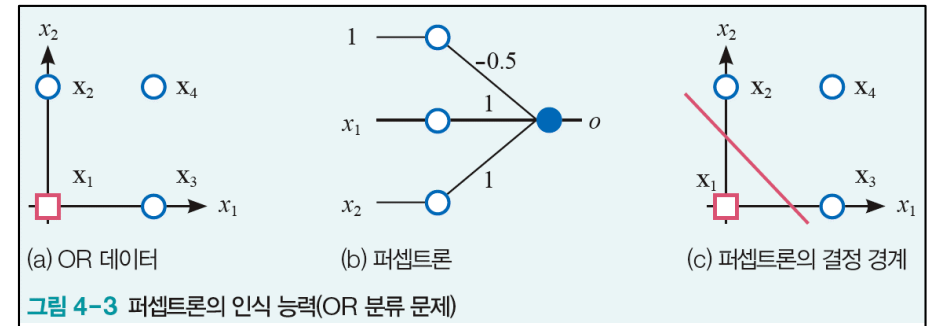


그림 4-3 퍼셉트론의 인식 능력(OR 분류 문제)

```
1 from sklearn.linear_model import Perceptron
2
3 # 훈련 집합 구축
4 X = [[0,0],[0,1],[1,0],[1,1]]
5 y = [-1,1,1,1]
6
7 # fit 함수로 Perceptron 학습
8 p = Perceptron()
9 p.fit(X,y)
10
11 print("학습된 퍼셉트론의 매개변수: ", p.coef_, p.intercept_)
12 print("훈련집합에 대한 예측: ", p.predict(X))
13 print("정확률 측정: ", p.score(X,y)*100, "%")
```

학습된 퍼셉트론의 매개변수: $\begin{bmatrix} 2. & 2. \end{bmatrix}$ $[-1.]$
훈련집합에 대한 예측: $[-1 \ 1 \ 1 \ 1]$
정확률 측정: 100.0 %

04행-05행 훈련 데이터셋 만들기

08행 Perceptron 함수를 호출해 객체 P를 생성

09행 fit 함수를 이용해 데이터 X와 y에 대한 학습 수행

11행 퍼셉트론의 매개변수 가중치와 바이어스 출력

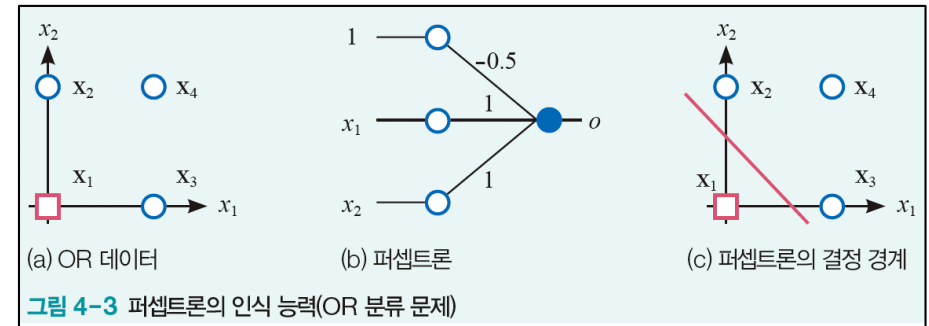
12행 predict 함수를 이용해 훈련집합 X를 테스트용으로 간주하고 예측 수행

13행 score함수는 X를 퍼셉트론으로 예측한 값과 y 레이블을 비교해 맞힌 샘플 개수를 세어 정확률을 계산

4.6 퍼셉트론 프로그래밍

4.6.1 OR 데이터 인식

- 》 실습 4-1. OR 데이터 퍼셉트론으로 분류하기



```
1 from sklearn.linear_model import Perceptron
2
3 # 훈련 집합 구축
4 X = [[0,0],[0,1],[1,0],[1,1]]
5 y = [-1,1,1,1]
6
7 # fit 함수로 Perceptron 학습
8 p = Perceptron()
9 p.fit(X,y)
10
11 print("학습된 퍼셉트론의 매개변수: ", p.coef_, p.intercept_)
12 print("훈련집합에 대한 예측: ", p.predict(X))
13 print("정확률 측정: ", p.score(X,y)*100, "%")
```

학습된 퍼셉트론의 매개변수: $\begin{bmatrix} 2. & 2. \end{bmatrix}$ $[-1.]$

훈련집합에 대한 예측: $[-1 \ 1 \ 1 \ 1]$

정확률 측정: 100.0 %

직선 방정식의 계수에 상수를 곱해도 같은 직선이므로

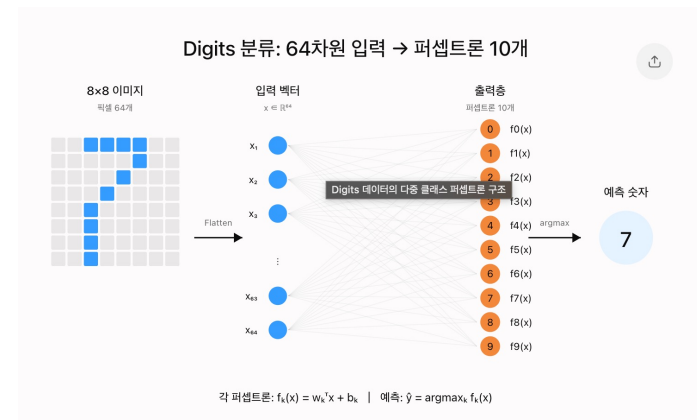
퍼셉트론 학습결과로 얻어진 결정 경계 $2x_1 + 2x_2 - 1 = 0$ 와

그림4-3의 결정 경계 $1x_1 + 1x_2 - 0.5 = 0$ 는 동일함을 알 수 있음

4.6 퍼셉트론 프로그래밍

4.6.2 필기 숫자 데이터 인식

- 》 실습 4-2. 필기 숫자 데이터 단층 퍼셉트론으로 분류하기



```
1 from sklearn import datasets
2 from sklearn.linear_model import Perceptron
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6 # 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 np.random.seed(0)
8 digit = datasets.load_digits()
9 x_train, x_test, y_train, y_test = train_test_split(digit.data, digit.target, train_size=0.6)
10
11 # fit 함수로 Perceptron 학습
12 p = Perceptron(max_iter=100, eta0=0.001, verbose=0)
13 # digit 데이터로 모델링
14 p.fit(x_train, y_train)
15 # 테스트 집합으로 예측
16 res = p.predict(x_test)
17
18 # 혼동 행렬
19 conf = np.zeros((10, 10))
20 for i in range(len(res)):
21     conf[res[i]][y_test[i]] += 1
22 print(conf)
23
24 # 정확률 계산
25 no_correct = 0
26 for i in range(10):
27     no_correct += conf[i][i]
28 accuracy = no_correct / len(res)
29 print("테스트 집합에 대한 정확률은 ", accuracy * 100, "%입니다.")
```

02행 sklearn의 linear_model 클래스가 제공하는 Perceptron 함수를 불러옴

09행 train_test_split 함수로 digit 데이터를 훈련 집합과 테스트 집합으로 분할

12행 Perceptron 객체를 생성해 p에 저장 (eta0 : learning rate), 가중치 벡터 10개

14행 훈련 집합으로 퍼셉트론을 학습

16행 테스트 집합에 대해 예측을 수행

19행-22행 혼동 행렬을 계산

25행-28행 정확률을 계산

```
[[60.  0.  0.  0.  0.  0.  1.  0.  0.  1.]
 [ 0. 70.  5.  0.  2.  2.  1.  2.  4.  3.]
 [ 0.  0. 64.  1.  0.  0.  0.  0.  1.  0.]
 [ 0.  0.  0. 54.  0.  0.  0.  0.  1.  0.]
 [ 0.  0.  0.  0. 60.  0.  0.  1.  0.  0.]
 [ 0.  0.  0.  3.  0. 86.  2.  0.  0.  3.]
 [ 0.  0.  0.  0.  0.  1. 71.  0.  0.  0.]
 [ 0.  0.  0.  1.  1.  0.  0. 62.  0.  0.]
 [ 0.  3.  2. 10.  0.  0.  1.  0. 72. 13.]
 [ 0.  0.  0.  1.  0.  0.  0.  0.  0. 54.]]
테스트 집합에 대한 정확률은 90.82058414464534 %입니다.
```

<https://www.kaggle.com/yukyungchoi/2023-ai-w3-p1>

4.6 퍼셉트론 프로그래밍

기계학습의 디자인 패턴을 기억하자

4.6.2 필기 숫자 데이터 인식

- 》 실습 4-2. 필기 숫자 데이터 단층 퍼셉트론으로 분류하기 vs 실습 3-5

》 실습 4-2. 필기 숫자 데이터에 퍼셉트론 적용

```
1 from sklearn import datasets
2 from sklearn.linear_model import Perceptron
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6 # 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 np.random.seed(0)
8 digit = datasets.load_digits()
9 x_train, x_test, y_train, y_test = train_test_split(digit.data, digit.target, train_size=0.6)
10
11 # fit 함수로 Perceptron 학습
12 p = Perceptron(max_iter=100, eta0=0.001, verbose=0)
13 # digit 데이터로 모델링
14 p.fit(x_train, y_train)
15 # 테스트 집합으로 예측
16 res = p.predict(x_test)
17
18 # 혼동 행렬
19 conf = np.zeros((10,10))
20 for i in range(len(res)):
21     conf[res[i]][y_test[i]]+=1
22 print(conf)
23
24 # 정확률 계산
25 no_correct=0
26 for i in range(10):
27     no_correct+=conf[i][i]
28 accuracy=no_correct/len(res)
29 print("테스트 집합에 대한 정확률은 ", accuracy*100, "%입니다.")
```

02행 sklearn의 linear_model 클래스가 제공하는 Perceptron 함수를 불러옴

09행 train_test_split 함수로 digit 데이터를 훈련 집합과 테스트 집합으로 분할

12행 Perceptron 객체를 생성해 p에 저장

14행 훈련 집합으로 퍼셉트론을 학습

16행 테스트 집합에 대해 예측을 수행

19행-22행 혼동 행렬을 계산

25행-28행 정확률을 계산

```
[[60.  0.  0.  0.  0.  0.  1.  0.  0.  1.]
 [ 0. 70.  5.  0.  2.  2.  1.  2.  4.  3.]
 [ 0.  0. 64.  1.  0.  0.  0.  0.  1.  0.]
 [ 0.  0.  0. 54.  0.  0.  0.  0.  1.  0.]
 [ 0.  0.  0.  0. 60.  0.  0.  1.  0.  0.]
 [ 0.  0.  0.  3.  0. 86.  2.  0.  0.  3.]
 [ 0.  0.  0.  0.  0.  1. 71.  0.  0.  0.]
 [ 0.  0.  0.  1.  1.  0.  0. 62.  0.  0.]
 [ 0.  3.  2. 10.  0.  0.  1.  0. 72. 13.]
 [ 0.  0.  0.  1.  0.  0.  0.  0.  0. 54.]]
테스트 집합에 대한 정확률은 90.82058414464534 %입니다.
```


4.6 퍼셉트론 프로그래밍

기계학습의 디자인 패턴을 기억하자

4.6.2 필기 숫자 데이터 인식

- » 실습 4-2. 필기 숫자 데이터에 단층 퍼셉트론 적용 vs 실습 3-5

» 실습 3-5. 필기 숫자 인식-훈련 집합으로 학습하고 테스트 집합으로 성능 측정 (혼동 행렬, 정확률)

```
1 from sklearn import datasets
2 from sklearn import svm
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6
7 # 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
8 np.random.seed(0)
9 digit = datasets.load_digits()
10 x_train, x_test, y_train, y_test = train_test_split(digit.data, digit.target, train_size=0.6)
11
12
13 # svm의 분류 모델 SVC를 학습
14 s=svm.SVC(gamma=0.001)
15 s.fit(x_train, y_train)
16 res = s.predict(x_test)
17
18
19 # 혼동 행렬 구함
20 conf=np.zeros((10,10))
21 for i in range(len(res)):
22     conf[res[i]][y_test[i]]+=1
23 print(conf)
24
25
26 # 정확률 측정하고 출력
27 no_correct=0
28 for i in range(10):
29     no_correct+=conf[i][i]
30 accuracy=no_correct/len(res)
31 print("테스트 집합에 대한 정확률은", accuracy*100, "%입니다.")
```

03행 `model_selection` 클래스가 제공하는 `train_test_split` 함수를 불러옴

08행 난수 생성 고정을 위해 사용 (`train_test_split` 함수가 난수를 사용)

10행 `train_test_split` 함수를 이용하여 `digit` 데이터를 훈련 집합과 테스트 집합으로 분할. 첫 번째와 두 번째 매개변수는 분할할 데이터의 특징 벡터와 레이블이며, 세 번째 매개변수는 훈련 집합의 비율

14~15행 훈련 집합으로 SVM을 학습

16행 테스트 집합으로 예측을 수행

20~22행 혼동 행렬 계산

27~30행 정확률을 계산

[	60.	0.	0.	0.	0.	0.	0.	0.	0.]
[	0.	73.	1.	0.	0.	0.	0.	1.	0.]
[	0.	0.	69.	0.	0.	0.	0.	0.	0.]
[	0.	0.	0.	70.	0.	0.	0.	0.	0.]
[	0.	0.	0.	0.	63.	0.	0.	0.	0.]
[	0.	0.	0.	0.	0.	87.	0.	0.	0.]
[	0.	0.	0.	0.	0.	1.	76.	0.	0.]
[	0.	0.	1.	0.	0.	0.	0.	65.	0.]
[	0.	0.	0.	0.	0.	0.	0.	77.	0.]
[	0.	0.	0.	0.	0.	1.	0.	0.	74.]

테스트 집합에 대한 정확률은 99.30458970792768 %입니다.

4.6 퍼셉트론 프로그래밍

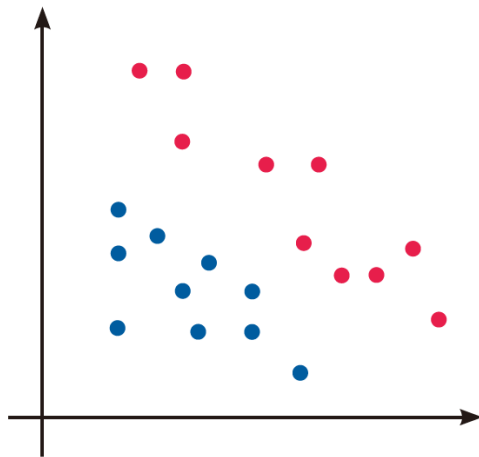
4.6.2 필기 숫자 데이터 인식

- >> 실습 4-2. 단일 퍼셉트론 vs. SVM
 - 퍼셉트론 정확률 : 90.82%
 - SVM 정확률 : 99.30%

→ 퍼셉트론은 1950년에 개발된 원시적인 신경망이고, SVM은 1990년대에 다층 퍼셉트론을 앞지른 발전된 모델이기 때문에 SVM 성능이 더욱 좋음

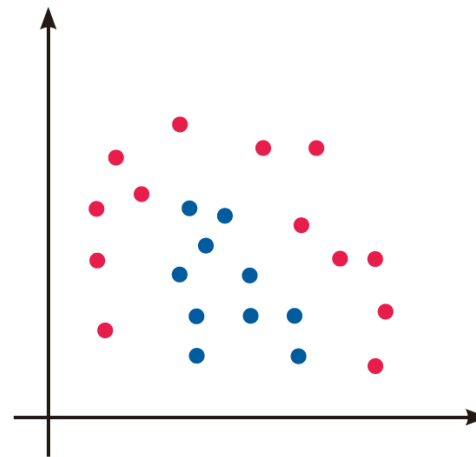
4.7 다층 퍼셉트론

- 단층 퍼셉트론은 **선형 분류기**라는 근본적인 한계가 있어 인식 오류를 피할 수 없음
- 다층 퍼셉트론은 은닉층을 추가하여 **비선형 분류기**로 확장 가능



(a) 선형 분리 가능

그림 4-9 데이터의 선형 분리 가능성



(b) 선형 분리 불가능

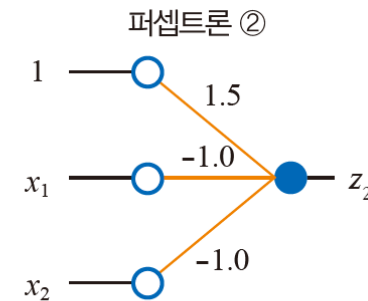
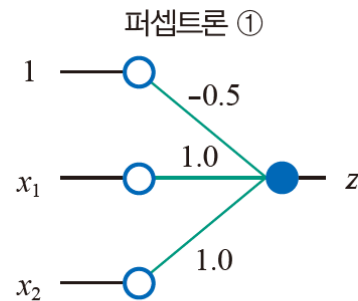
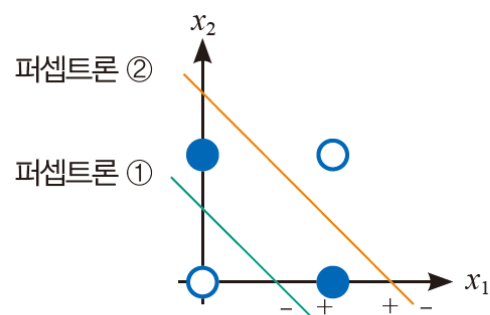


다층 퍼셉트론으로 해결해보자

4.7 다층 퍼셉트론

4.7.1 특징 공간 변환

- 퍼셉트론 2개를 연속적으로 사용하면 특징 공간을 3개의 부분공간으로 나눌 수 있음
- 선형 분리 불가능한 XOR 문제를 풀 수 있는 실마리



(a) 퍼셉트론 2개를 이용해 부분공간 3개로 분할 (b) 2개의 퍼셉트론

그림 4-10 퍼셉트론 2개로 XOR 데이터를 인식하는 길을 찾음

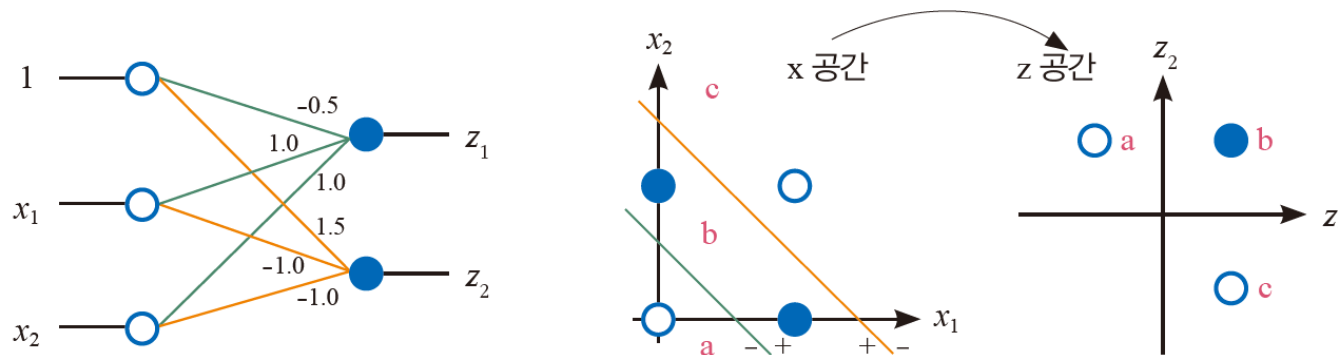
4.7 다층 퍼셉트론

4.7.1 특징 공간 변환

- 두 퍼셉트론을 결합한 병렬구조를 통해

원래의 특징 공간 (x_1, x_2) 를 새로운 특징 공간 (z_1, z_2) 로 변환함

- 원래 공간에서 선형 분리가 불가능하던 4개의 점이 새로운 공간에서는 선형 분리가 가능해짐



(a) 공간 변환하는 퍼셉트론 2개의 병렬 결합

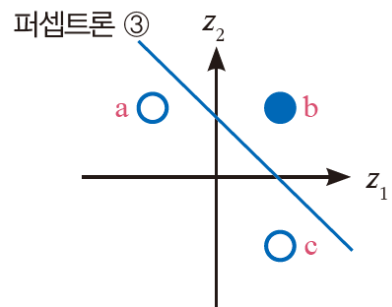
(b) 특징 공간 변환(세 부분공간을 세 점으로 매핑)

그림 4-11 퍼셉트론 2개를 병렬 결합해 특징 공간을 변환

4.7 다층 퍼셉트론

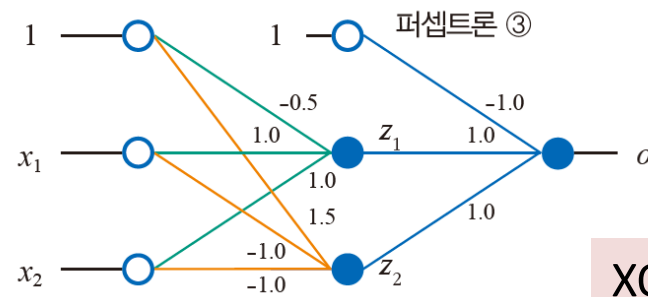
4.7.1 특징 공간 변환

- 퍼셉트론 하나를 더 사용하면 XOR 분류 문제를 푸는 신경망을 만들 수 있음
- [그림 4-12(a)]는 새로운 특징 공간(z_1, z_2)에서 분류를 수행하는 퍼셉트론을 보여줌
- [그림 4-11(a)]의 신경망에 [그림 4-12(a)]의 퍼셉트론을 순차적으로 덧붙이면 [그림 4-12(b)]이 신경망이 되며, 이 신경망을 **다층 퍼셉트론 multi-layer perceptron** 이라고 함



(a) 공간 분할하는 세 번째 퍼셉트론

그림 4-12 다층 퍼셉트론

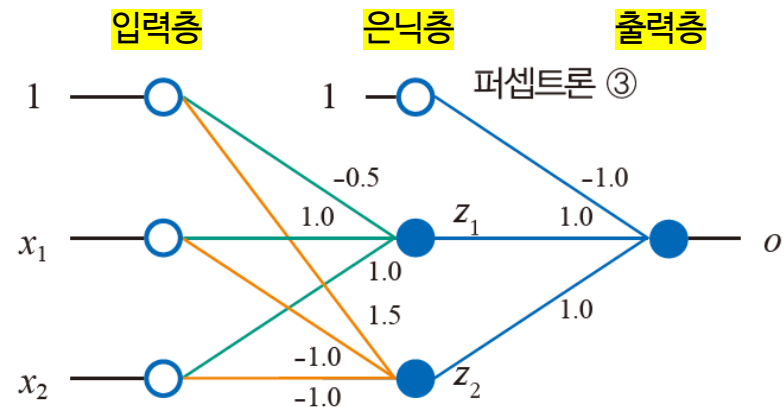
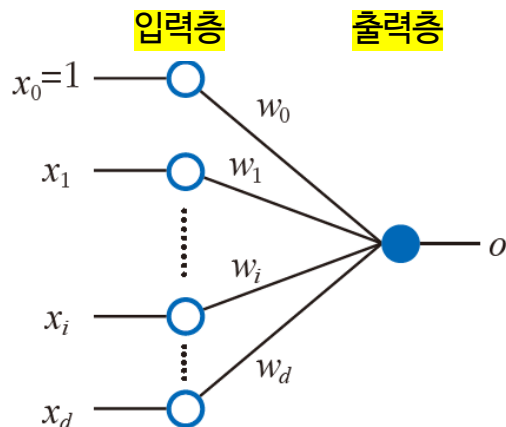


(b) 세 번째 퍼셉트론을 순차적으로 덧붙임

XOR문제 해결

4.7 다층 퍼셉트론

- 4.7.1 특징 공간 변환
 - 신경망을 <특징 공간 변환기>로 바라 볼 수 있음
 - 다층 퍼셉트론에서는 원래 공간에서 선형 분리 불가능하던 데이터 분포가 “은닉 공간”에서는 선형 분리 가능한 분포로 바뀌었음
 - 아래 예시에서는 다층 퍼셉트론에서는 은닉층을 하나만 사용했으나, 영상이나 자연어와 같이 매우 복잡한 데이터는 하나의 은닉층으로 충분한 특징 변환이 불가능하여 여러 은닉층을 사용함
 - 어려운 문제일 수록, 깊은 신경망을 사용한다는 의미!



4.7 다층 퍼셉트론

4.7.2 다층 퍼셉트론의 구조

- [그림 4-13]은 신경망을 일반화한 다층 퍼셉트론임
- 다층 퍼셉트론은 입력층, 은닉층, 출력층으로 구성되는데 층을 연결하는 **가중치** **뭉치가 두 군데에 있으므로 층이 3개가 아니라 2개라고 간주함** : MLP에서 층의 수는 가중치 뭉치의 수
- 따라서 [그림4-2(a)]의 신경망은 층이 하나라고 간주하여 단층 퍼셉트론 single-layer perceptron, [그림4-13]의 신경망은 **다층 퍼셉트론 MLP: multi-layer perceptron** 으로 구별해 부름
- 단층 퍼셉트론은 퍼셉트론으로 줄여 부름

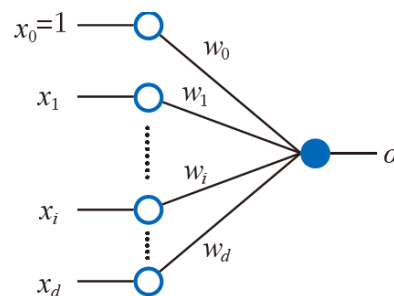


그림 4-2(a)

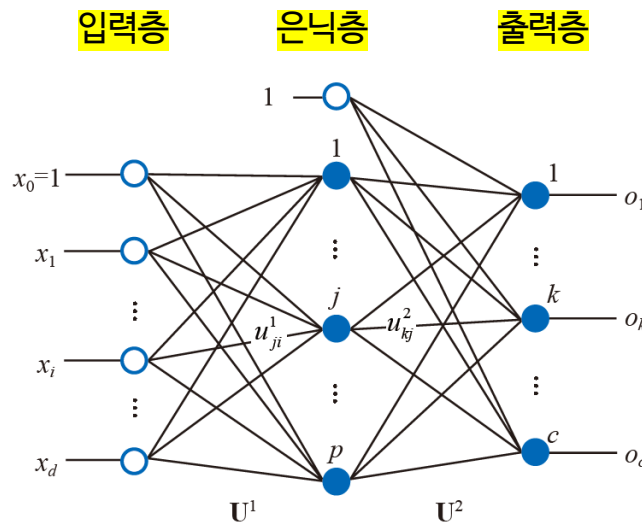


그림 4-13 다층 퍼셉트론의 구조

4.7 다층 퍼셉트론

- 4.7.2 다층 퍼셉트론의 구조
 - 단층 퍼셉트론의 출력층은 분류기
 - 데이터를 기존 입력 공간 그대로 사용
 - Feature mapping 없이 곧바로 분류
 - 다층 퍼셉트론의 은닉층은 자동 특징 추출기, 출력층은 분류기
 - 은닉층: 비선형 활성화 함수를 통해 입력 데이터를 고차원 feature로 변환
 - 출력층: 변환된 feature를 사용해 최종 분류기 학습

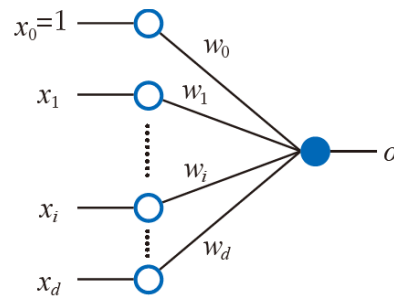


그림 4-2(a)

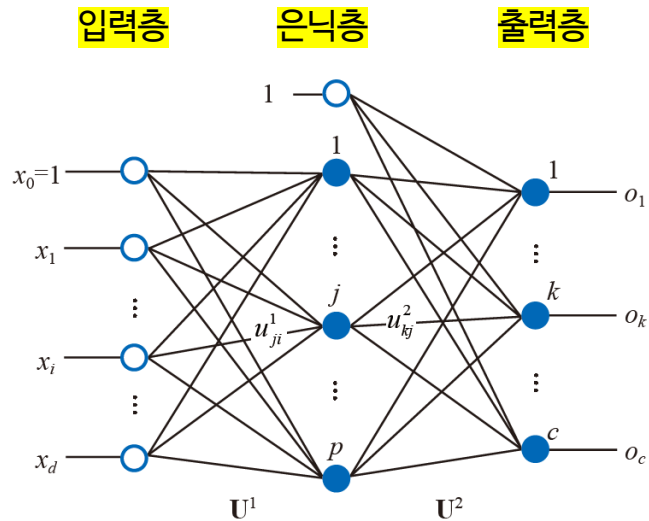


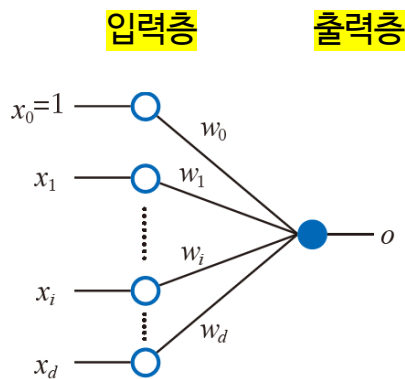
그림 4-13 다층 퍼셉트론의 구조

4.7 다층 퍼셉트론

- 4.7.2 다층 퍼셉트론의 구조
 - 데이터가 주어지면 입력층의 노드 개수와 출력층의 노드 개수는 자동으로 정해짐
 - (예) iris의 경우 특징이 4개이므로 바이어스 노드를 포함해 입력층의 노드는 5개이고, 부류가 3개이므로 출력층의 노드는 3개임
 - 은닉층의 노드 개수는 신경망을 설계하는 사람이 정해야 하는 하이퍼 매개변수임
 - 은닉층의 노드 개수가 많으면 신경망의 용량이 커져서 복잡한 데이터 모델링에 유리하지만 학습이 오래 걸리고 과잉 적합이 나타날 가능성이 커짐

4.7 다층 퍼셉트론

- 4.7.2 다층 퍼셉트론의 구조
 - 가중치 집합이 두 곳 이상 존재하면 다층 퍼셉트론
 - 아래 예시는 가중치 집단이 2곳
 - 입력층과 은닉층을 연결하는 가중치 집합 U^1 (Feature mapper)
 - 은닉층과 출력층을 연결하는 가중치 집합 U^2 (Classifier)



단층 퍼셉트론의 구조

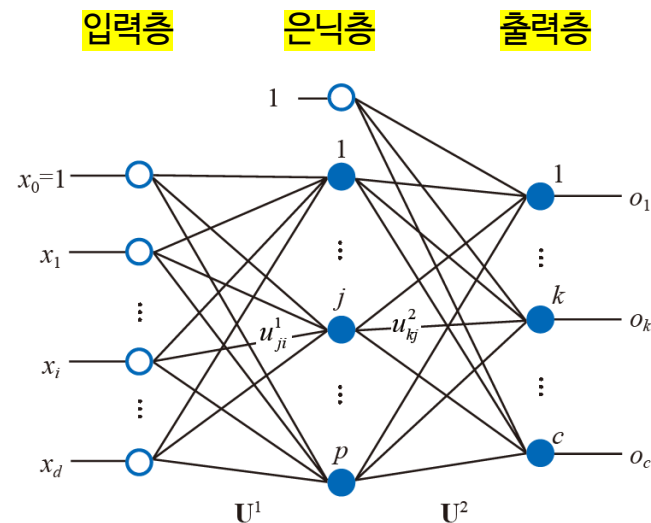


그림 4-13 다층 퍼셉트론의 구조

4.7 다층 퍼셉트론

- 4.7.2 다층 퍼셉트론의 구조
 - 가중치 집합이 두 곳 이상 존재하면 다층 퍼셉트론
 - 아래 예시는 가중치 집단이 2곳
 - 입력층과 은닉층을 연결하는 가중치 집합 U^1 (Feature mapper)
 - 은닉층과 출력층을 연결하는 가중치 집합 U^2 (Classifier)

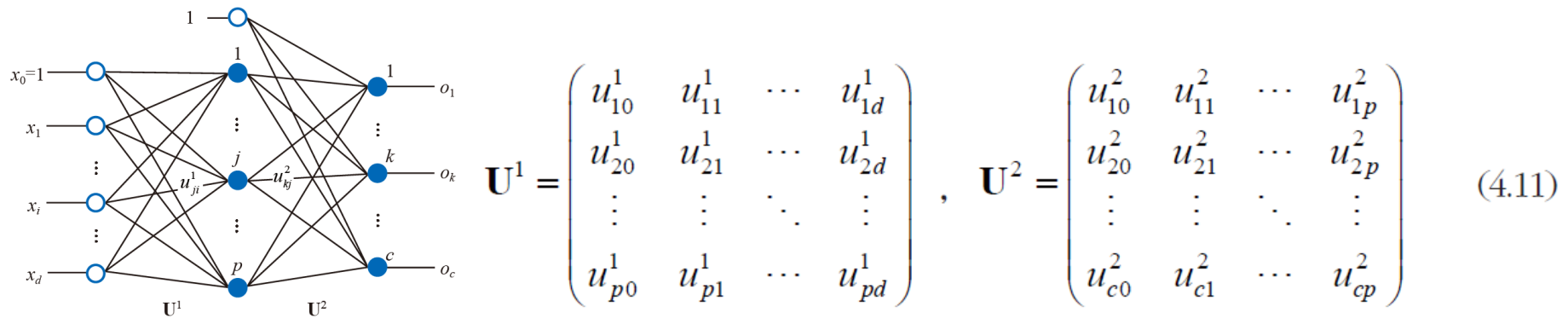


그림 4-13 다층 퍼셉트론의 구조

u_{ji}^1 은 i 번째 입력 노드와 j 번째 은닉 노드를 연결하는 가중치
 u_{kj}^2 는 j 번째 은닉 노드와 k 번째 출력 노드를 연결하는 가중치

4.7 다층 퍼셉트론

4.7.3 다층 퍼셉트론의 동작

- 식(4.12)는 입력된 특징 벡터 x 를 은닉 공간의 특징 벡터 z 로 변환하는 $x \rightarrow z$ 를 수행

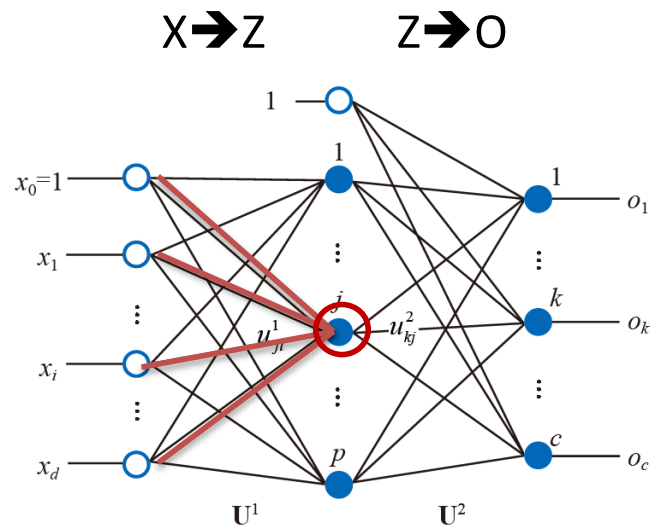


그림 4-13 다층 퍼셉트론의 구조

j 번째 은닉 노드(z_j)의 연산:

$$z_j = \tau_1(s_j), j = 1, 2, \dots, p \quad \text{식(4.12)}$$

이때 $s_j = u_j^1 x^T$ 이고,

$$u_j^1 = (u_{j0}^1, u_{j1}^1, \dots, u_{jd}^1), x = (1, x_1, x_2, \dots, x_d)$$

4.7 다층 퍼셉트론

4.7.3 다층 퍼셉트론의 동작

- 식(4.13)은 변환된 특징 벡터 z 를 출력 벡터 o 로 변환하는 $z \rightarrow o$ 를 수행

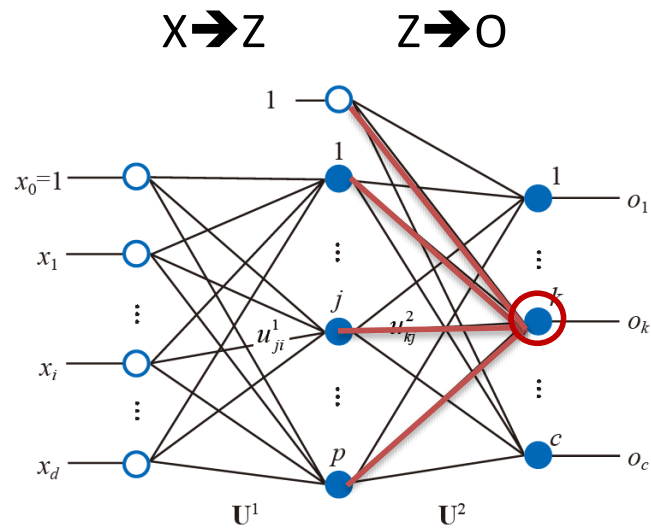


그림 4-13 다층 퍼셉트론의 구조

k 번째 출력 노드(o_k)의 연산:

$$o_k = \tau_2(s_k), k = 1, 2, \dots, c$$

식(4.13)

이때 $s_k = u_k^2 z^T$ 이고,

$$u_k^2 = (u_{k0}^2, u_{k1}^2, \dots, u_{kp}^2), z = (1, z_1, z_2, \dots, z_p)$$

4.7 다층 퍼셉트론

4.7.3 다층 퍼셉트론의 동작

- 식(4.14)는 전체 과정을 행렬 표기로 간결하게 표현한 것임
- 활성함수를 τ_1 과 τ_2 로 구분한 이유는 일반적으로 은닉층과 출력층이 서로 다른 활성 함수를 사용하기 때문
- 분류문제에서 주로 은닉층은 ReLU, 출력층은 softmax를 활성 함수로 사용함

$$\mathbf{o} = \tau_2(\mathbf{U}^2 \tau_1(\mathbf{U}^1 \mathbf{x}^T))$$

식(4.14)

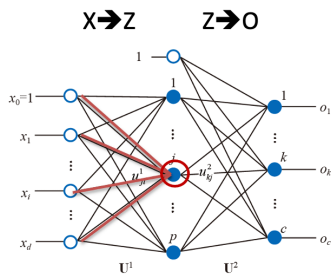


그림 4-13 다층 퍼셉트론의 구조

j 번째 은닉 노드(z_j)의 연산:

$$z_j = \tau_1(s_j), j = 1, 2, \dots, p$$

이때 $s_j = \mathbf{u}_j^1 \mathbf{x}^T$ 이고,
 $\mathbf{u}_j^1 = (u_{j0}^1, u_{j1}^1, \dots, u_{jd}^1), \mathbf{x} = (1, x_1, x_2, \dots, x_d)$

식(4.12)

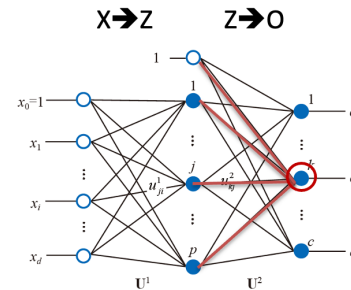


그림 4-13 다층 퍼셉트론의 구조

k 번째 출력 노드(o_k)의 연산:

$$o_k = \tau_2(s_k), k = 1, 2, \dots, c$$

이때 $s_k = \mathbf{u}_k^2 \mathbf{z}^T$ 이고,
 $\mathbf{u}_k^2 = (u_{k0}^2, u_{k1}^2, \dots, u_{kp}^2), \mathbf{z} = (1, z_1, z_2, \dots, z_p)$

식(4.13)

4.7 다층 퍼셉트론

4.7.3 다층 퍼셉트론의 동작

- 식(4.14)는 샘플 하나 $\mathbf{x}$ 가 입력되었을 때 다층 퍼셉트론의 동작을 수식으로 표현한 것
- 여러 샘플로 구성된 훈련 집합/테스트 집합을 한꺼번에 처리하는 방식으로 수식을 변경 필요
- 식(4.15)는 n 개의 샘플로 구성된 데이터 집합 $\mathbf{X}$ 를 정의함
- N 개의 샘플로 구성된 데이터 집합을 한꺼번에 처리하는 수식을 식(4.16)로 쓸 수 있음**

$$\mathbf{o} = \tau_2(\mathbf{U}^2 \tau_1(\mathbf{U}^1 \mathbf{x}^T)) \quad \text{식(4.14)}$$

$$\mathbf{O} = \tau_2(\mathbf{U}^2 \tau_1(\mathbf{U}^1 \mathbf{X}^T)) \quad \text{식(4.16)}$$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & \cdots & x_d^1 \\ 1 & x_1^2 & \cdots & x_d^2 \\ 1 & \vdots & \ddots & \vdots \\ 1 & x_1^n & \cdots & x_d^n \end{pmatrix} \quad \text{식(4.15)}$$

4.7 다층 퍼셉트론

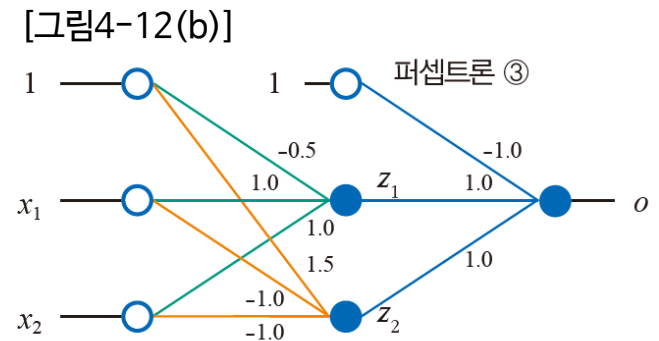
4.7.3 다층 퍼셉트론의 동작

[예제 4-3] 다층 퍼셉트론의 동작

[그림4-12(b)]에 있는 다층 퍼셉트론의 가중치 행렬을 식(4.11)처럼 쓰면 다음과 같음 ($d=2, p=2, c=1$)

$$\mathbf{U}^1 = \begin{pmatrix} -0.5 & 1.0 & 1.0 \\ 1.5 & -1.0 & -1.0 \end{pmatrix}, \mathbf{U}^2 = \begin{pmatrix} -1 & 1.0 & 1.0 \end{pmatrix}$$

d=입력 노드 수
p=은닉 노드 수
c=출력 노드 수



$$\begin{aligned} \mathbf{O} &= \tau_2 \left((-1 \ 1.0 \ 1.0) \tau_1 \left(\begin{pmatrix} -0.5 & 1.0 & 1.0 \\ 1.5 & -1.0 & -1.0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \right) \right) \\ &= \tau_2 \left((-1 \ 1.0 \ 1.0) \tau_1 \left(\begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \right) \right) \\ &= \tau_2 \left((-1 \ 1.0 \ 1.0) \begin{pmatrix} 1 \\ 1.0 \\ 1.0 \end{pmatrix} \right) \\ &= \tau_2 ((1)) = 1 \end{aligned}$$

$x \rightarrow (0,1)$ 데이터 샘플 한 개 입력 예시

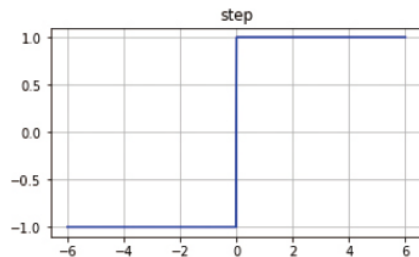
$$\begin{aligned} \mathbf{O} &= \tau_2 \left((-1 \ 1.0 \ 1.0) \tau_1 \left(\begin{pmatrix} -0.5 & 1.0 & 1.0 \\ 1.5 & -1.0 & -1.0 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix} \right) \right) \\ &= \tau_2 \left((-1 \ 1.0 \ 1.0) \tau_1 \left(\begin{pmatrix} -0.5 & 0.5 & 0.5 & 1.5 \\ 1.5 & 0.5 & 0.5 & -0.5 \end{pmatrix} \right) \right) \\ &= \tau_2 \left((-1 \ 1.0 \ 1.0) \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1.0 & 1.0 & 1.0 & 1.0 \\ 1.0 & 1.0 & 1.0 & -1.0 \end{pmatrix} \right) \\ &= \tau_2 ((-1 \ 1 \ 1 \ -1)) = (-1 \ 1 \ 1 \ -1) \end{aligned}$$

$X \rightarrow$ XOR 데이터 전부 입력 예시

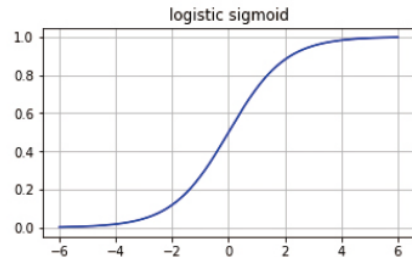
4.7 다층 퍼셉트론

4.7.4 활성화 함수

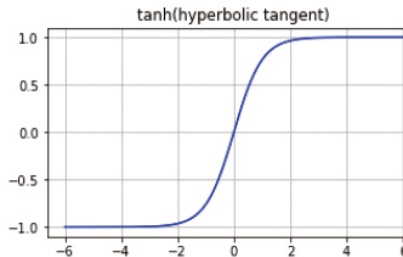
- 단층 퍼셉트론은 [그림4-14(a)]의 보통 계단 함수를 활성화 함수로 사용함
 - 이 함수는 1과 -1의 두 가지 상태를 출력하는데, 세상에는 두 가지 상태로 표현할 수 없는 상황이 많음
- 다층 퍼셉트론이 신경망 연구의 주류였던 1980~1990년대에는 주로 시그모이드 함수를 사용함
- ReLU는 뒤늦게 발견되었으며, 딥러닝은 주로 ReLU를 사용함



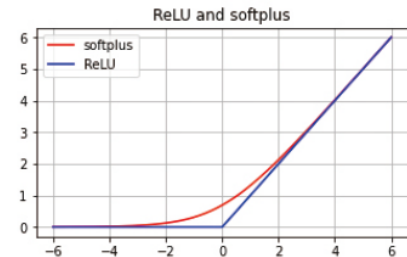
(a) 계단 함수



(b) 로지스틱 시그모이드



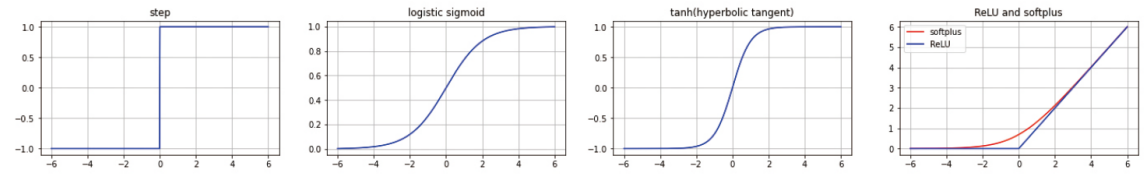
(c) 하이퍼볼릭 탄젠트 시그모이드



(d) 소프트플러스와 ReLU

그림 4-14 신경망이 사용하는 다양한 활성화 함수

4.7 다층 퍼셉트론



(a) 계단 함수

(b) 로지스틱 시그모이드

(c) 하이퍼볼릭 탄젠트 시그모이드

(d) 소프트플러스와 ReLU

그림 4-14 신경망이 사용하는 다양한 활성화 함수

4.7.4 활성화 함수

- **시그모이드 함수**는 신경망이 깊어질 수록 그레디언트 소멸 gradient vanishing 문제가 발생함
- **ReLU**는 시그모이드의 그레디언트 소멸 문제를 완화시켜줌

표 4-1 신경망에서 사용하는 여러 가지 활성화 함수

함수 이름	함수	1차 도함수	범위
계단	$\tau(s) = \begin{cases} 1 & s \geq 0 \\ -1 & s < 0 \end{cases}$	$\tau'(s) = \begin{cases} 0 & s \neq 0 \\ \text{불가} & s = 0 \end{cases}$	-1과 1
로지스틱 시그모이드	$\tau(s) = \frac{1}{1 + e^{-as}}$	$\tau'(s) = a\tau(s)(1 - \tau(s))$	(0,1)
하이퍼볼릭 탄젠트 (tanh) 시그모이드	$\tau(s) = \frac{2}{1 + e^{-as}} - 1$	$\tau'(s) = \frac{a}{2}(1 - \tau(s)^2)$	(-1,1)
소프트플러스	$\tau(s) = \log_e(1 + e^s)$	$\tau'(s) = \frac{1}{1 + e^{-s}}$	(0,∞)
렉티파이어(ReLU)	$\tau(s) = \max(0, s)$	$\tau'(s) = \begin{cases} 0 & s < 0 \\ 1 & s > 0 \\ \text{불가} & s = 0 \end{cases}$	(0,∞)

4.7 다층 퍼셉트론

4.7.4 활성화 함수

- 신경망의 출력층에서 주로 사용하는 활성화 함수로 softmax가 있음
 - 식(4.17)로 정의됨
- 대부분의 활성화 함수는 개별 노드에 독립적으로 적용
 - [표4-1] 계단함수, 로지스틱 시그모이드, 하이퍼볼릭 탄젠트 시그모이드, ReLU
- 반면, softmax는 c개의 노드를 같이 고려하여 계산한다는 점이 다름
 - 출력이 각 클래스별 확률 값을 의미하며, 모든 출력 값의 합이 1임

$$o_k = \frac{e^{s_k}}{\sum_{i=1,c} e^{s_i}} \quad \text{식(4.17)}$$

4.8 오류 역전파 알고리즘

- 다층 퍼셉트론은 오류 역전파 알고리즘으로 학습함
- 단일 퍼셉트론과 달리 은닉층이 있고, 활성화 함수가 시그모이드 이므로, 단일 퍼셉트론 학습 알고리즘보다 복잡

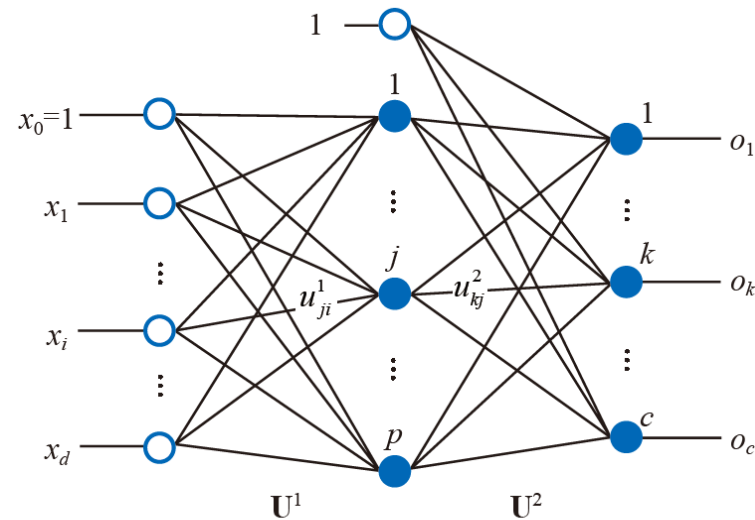
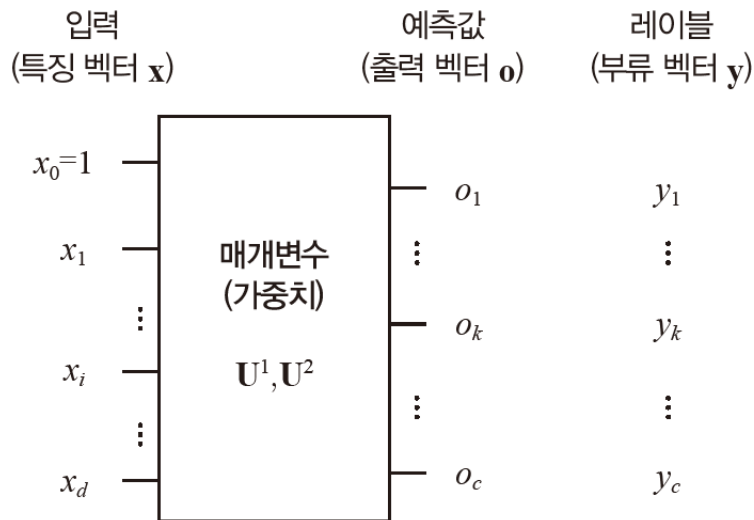


그림 4-13 다층 퍼셉트론의 구조

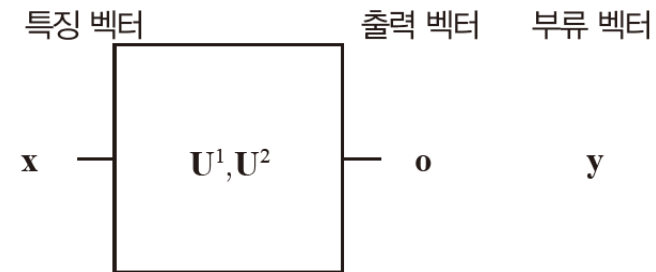
$$\mathbf{U}^1 = \begin{pmatrix} u_{10}^1 & u_{11}^1 & \cdots & u_{1d}^1 \\ u_{20}^1 & u_{21}^1 & \cdots & u_{2d}^1 \\ \vdots & \vdots & \ddots & \vdots \\ u_{p0}^1 & u_{p1}^1 & \cdots & u_{pd}^1 \end{pmatrix}, \quad \mathbf{U}^2 = \begin{pmatrix} u_{10}^2 & u_{11}^2 & \cdots & u_{1p}^2 \\ u_{20}^2 & u_{21}^2 & \cdots & u_{2p}^2 \\ \vdots & \vdots & \ddots & \vdots \\ u_{c0}^2 & u_{c1}^2 & \cdots & u_{cp}^2 \end{pmatrix} \quad (4.11)$$

4.8 오류 역전파 알고리즘

- 4.8.1 손실함수 설계
 - 다층 퍼셉트론의 블록 다이어그램
 - 신경망의 출력 벡터(예측값) $\mathbf{o}$ 가 부류 벡터(참값) $\mathbf{y}$ 와 같을수록 매개변수 $U1$ 과 $U2$ 는 데이터를 잘 인식. 다르면 $\mathbf{o}$ 와 $\mathbf{y}$ 가 가까워지도록 $U1$ 과 $U2$ 를 갱신해야 함. 오차가 클수록 갱신하는 양이 큼



(a) 입력과 출력, 기댓값의 관계



(b) 벡터 표현

그림 4-15 다층 퍼셉트론의 블록 다이어그램

4.8 오류 역전파 알고리즘

4.8.1 손실함수 설계

- 샘플 하나의 오차를 측정하는 손실 함수

- 보통 y 는 원핫 코드로 표현. 예) 숫자 0이면 $y=(1,0,0,\dots,0)$, 2면 $y=(0,0,1,0,\dots,0)$

$$J(\mathbf{U}^1, \mathbf{U}^2) = \sqrt{(y_1 - o_1)^2 + (y_2 - o_2)^2 + \dots + (y_c - o_c)^2} = \|\mathbf{y} - \mathbf{o}\| \quad (4.18)$$

- 식 (4.18)을 확장

- 계산 편의를 위해 제곱을 하여 제곱근 제거
- 미니 배치 단위로 처리(샘플의 오차를 평균)

$$\begin{aligned} J(\mathbf{U}^1, \mathbf{U}^2) &= \frac{1}{|M|} \sum_{\mathbf{x} \in M} \|\mathbf{y} - \mathbf{o}\|^2 \\ &= \frac{1}{|M|} \sum_{\mathbf{x} \in M} \left\| \mathbf{y} - \tau_2 \left(\mathbf{U}^2 \tau_1 \left(\mathbf{U}^1 \mathbf{x}^T \right) \right) \right\|^2 \end{aligned} \quad (4.19)$$

4.8 오류 역전파 알고리즘

- 4.8.1 손실함수 설계
 - 가중치 갱신 (다층 퍼셉트론 학습)
 - 다층 퍼셉트론은 입력층과 은닉층 사이의 가중치 행렬 U_1 , 은닉층과 출력층 사이의 가중치 행렬 U_2 가 최적화할 매개변수 임
 - 가중치 갱신 규칙은 (4.20)과 같이 쓸 수 있음

$$\begin{aligned}
 \mathbf{U}^2 &= \mathbf{U}^2 + \rho(-\nabla \mathbf{U}^2) & \mathbf{U}^1 &= \mathbf{U}^1 + \rho(-\nabla \mathbf{U}^1) \\
 \text{학습률} & \quad \quad \quad \uparrow & & \\
 \text{오차의 미분값} & \quad \quad \uparrow & & \\
 \nabla \mathbf{U}^2 &= \begin{pmatrix} \frac{\partial J}{\partial u_{10}^2} & \frac{\partial J}{\partial u_{11}^2} & \cdots & \frac{\partial J}{\partial u_{1p}^2} \\ \frac{\partial J}{\partial u_{20}^2} & \frac{\partial J}{\partial u_{21}^2} & \cdots & \frac{\partial J}{\partial u_{2p}^2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial J}{\partial u_{c0}^2} & \frac{\partial J}{\partial u_{c1}^2} & \cdots & \frac{\partial J}{\partial u_{cp}^2} \end{pmatrix} & \nabla \mathbf{U}^1 &= \begin{pmatrix} \frac{\partial J}{\partial u_{10}^1} & \frac{\partial J}{\partial u_{11}^1} & \cdots & \frac{\partial J}{\partial u_{1d}^1} \\ \frac{\partial J}{\partial u_{20}^1} & \frac{\partial J}{\partial u_{21}^1} & \cdots & \frac{\partial J}{\partial u_{2d}^1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial J}{\partial u_{p0}^1} & \frac{\partial J}{\partial u_{p1}^1} & \cdots & \frac{\partial J}{\partial u_{pd}^1} \end{pmatrix}
 \end{aligned} \tag{4.20}$$

4.8 오류 역전파 알고리즘

- 4.8.1 손실함수 설계
 - 오류 역전파 error-backpropagation 학습 알고리즘
 - 출력층에서 시작하여 **역방향으로 오류를 전파한다**는 뜻에서 **오류 역전파**라 부름
 - 오차에 따라 가중치를 갱신하는데, U_2 를 먼저 갱신한 후에 U_1 을 갱신

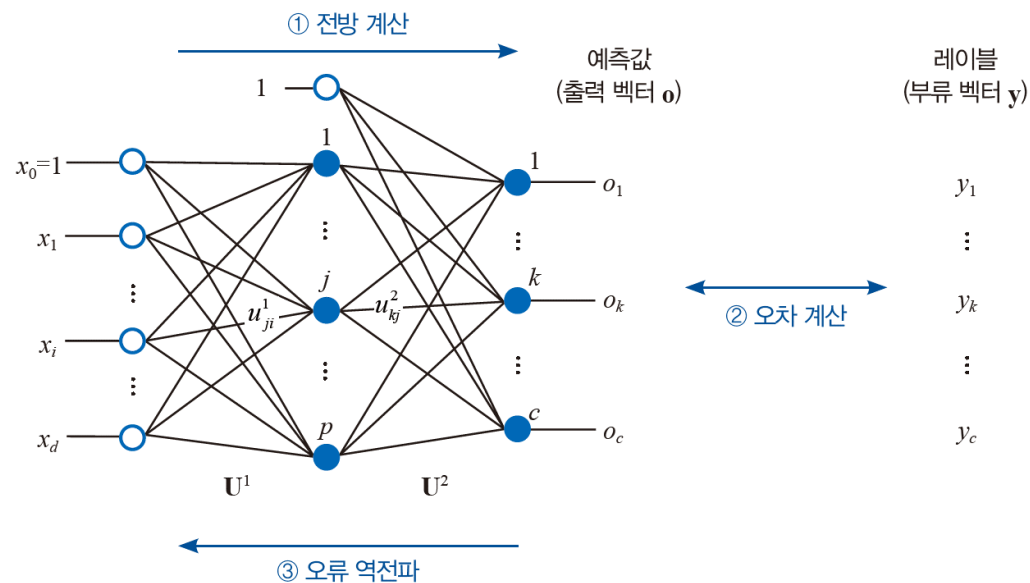


그림 4-16 오류 역전파 학습 알고리즘

4.9 다층 퍼셉트론 프로그래밍

4.9.1 sklearn의 필기 숫자 데이터셋

- >> 실습 4-3. 다층 퍼셉트론을 이용하여 필기 숫자 분류 하기

```
1 from sklearn import datasets
2 from sklearn.neural_network import MLPClassifier
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6 # 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 digit = datasets.load_digits()
8 x_train, x_test, y_train, y_test = train_test_split(digit.data, digit.target, train_size=0.6)
9
10 # MLP 분류기 모델을 학습
11 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001, batch_size=32, max_iter=300, solver='sgd', verbose=True)
12 mlp.fit(x_train, y_train)
13
14 # 테스트 집합으로 예측
15 res = mlp.predict(x_test)
16
17 # 혼동 행렬
18 conf = np.zeros((10,10))
19 for i in range(len(res)):
20     conf[res[i]][y_test[i]]+=1
21 print(conf)
22
23 # 정확률 계산
24 no_correct=0
25 for i in range(10):
26     no_correct+=conf[i][i]
27 accuracy=no_correct/len(res)
28 print("테스트 집합에 대한 정확률은", accuracy*100, "%입니다")
29
```

02행 sklearn의 neural_network 클래스가 제공하는 MLPClassifier 함수를 불러옴

08행 digit 데이터를 훈련 집합과 테스트 집합으로 분할 (학습:테스트=0.6:0.4)

12행 MLPClassifier 객체를 생성해 mlp에 저장

13행 훈련집합으로 다층 퍼셉트론을 학습

16행 테스트 집합에 대해 예측을 수행

<https://www.kaggle.com/yukyungchoi/2023-ai-w3-p3>

4.9 다층 퍼셉트론 프로그래밍

4.9.1 sklearn의 필기 숫자 데이터셋

- >> 실습 4-3. 다층 퍼셉트론을 이용하여 필기 숫자 분류 하기

```
1 from sklearn import datasets
2 from sklearn.neural_network import MLPClassifier
3 from sklearn.model_selection import train_test_split
4 import numpy as np
5
6 # 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 digit = datasets.load_digits()
8 x_train, x_test, y_train, y_test = train_test_split(digit.data, digit.target)
9
10
11 #MLP 분류기 모델을 학습
12 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001)
13 mlp.fit(x_train, y_train)
14
15 # 테스트 집합으로 예측
16 res = mlp.predict(x_test)
17
18 # 혼동 행렬
19 conf = np.zeros((10,10))
20 for i in range(len(res)):
21     conf[res[i]][y_test[i]]+=1
22 print(conf)
23
24 # 정확률 계산
25 no_correct=0
26 for i in range(10):
27     no_correct+=conf[i][i]
28 accuracy=no_correct/len(res)
29 print("테스트 집합에 대한 정확률은", accuracy*100, "%입니다")
```

```
Iteration 80, loss = 0.00560473
Iteration 81, loss = 0.00555686
Iteration 82, loss = 0.00543775
Iteration 83, loss = 0.00539543
Iteration 84, loss = 0.00531454
Iteration 85, loss = 0.00521296
Iteration 86, loss = 0.00517708
Iteration 87, loss = 0.00507465
Iteration 88, loss = 0.00510139
Iteration 89, loss = 0.00495557
Iteration 90, loss = 0.00493567
Iteration 91, loss = 0.00489034
Iteration 92, loss = 0.00480490
Iteration 93, loss = 0.00475899
Iteration 94, loss = 0.00470473
Iteration 95, loss = 0.00470833
Iteration 96, loss = 0.00462978
Iteration 97, loss = 0.00460594
Iteration 98, loss = 0.00449170
Iteration 99, loss = 0.00450599
Iteration 100, loss = 0.00441188
Iteration 101, loss = 0.00436080
Iteration 102, loss = 0.00433916
Iteration 103, loss = 0.00427511
Iteration 104, loss = 0.00425572
Iteration 105, loss = 0.00421061
Iteration 106, loss = 0.00418496
Iteration 107, loss = 0.00412109
Iteration 108, loss = 0.00408352
Iteration 109, loss = 0.00402457
Training loss did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
[[75.  0.  0.  0.  0.  0.  1.  0.  0.  0.]
 [ 1. 62.  1.  0.  0.  0.  0.  0.  2.  0.]
 [ 1.  0. 67.  0.  0.  1.  0.  0.  1.  0.]
 [ 0.  0.  0. 80.  0.  0.  0.  0.  0.  1.]
 [ 0.  0.  0.  0. 76.  0.  1.  0.  1.  0.]
 [ 3.  0.  0.  1.  0. 73.  0.  0.  0.  1.]
 [ 0.  1.  0.  0.  0.  1. 68.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. 72.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  1.  0. 56.  1.]
 [ 0.  1.  0.  0.  0.  1.  0.  0.  0. 68.]]
테스트 집합에 대한 정확률은 96.94019471488178 %입니다
```

퍼셉트론의 성능보다는 높고,
SVM의 성능에 비해서는 낮음을 알 수 있음

4.9 다층 퍼셉트론 프로그래밍

4.9.2 MNIST의 데이터셋으로 확장하기

▪ >> 실습 4-4. 다층 퍼셉트론으로 MNIST 데이터셋 분류하기

```
1 from sklearn.datasets import fetch_openml
2 from sklearn.neural_network import MLPClassifier
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # MNIST 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 mnist=fetch_openml('mnist_784')
8 mnist.data=mnist.data/255.0
9 x_train=mnist.data[:60000]; x_test=mnist.data[60000:]
10 y_train=np.int16(mnist.target[:60000]); y_test=np.int16(mnist.target[60000:])
11
12 #MLP 분류기 모델을 학습
13 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001, batch_size=512, max_iter=300, solver='adam', verbose=True)
14 mlp.fit(x_train, y_train)
15
16 # 테스트 집합으로 예측
17 res = mlp.predict(x_test)
18
19 # 혼동 행렬
20 conf = np.zeros((10,10), dtype=np.int16)
21 for i in range(len(res)):
22     conf[res[i]][y_test[i]]+=1
23 print(conf)
24
25 # 정확률 계산
26 no_correct=0
27 for i in range(10):
28     no_correct+=conf[i][i]
29 accuracy=no_correct/len(res)
30 print("테스트 집합에 대한 정확률은", accuracy*100, "%입니다")
```

08행 [0,255] 범위의 값을 가진 mnist의 맵을 [0,1]로 정규화 함
09행 100000개의 데이터 중 6000개는 학습, 4000개는 테스트로 나눔
10행 mnist.target이 str형이므로 np.int16을 적용해 16비트 정수형으로 변환함

13행 미니배치 크기를 비교적 큰 값인 512로 설정
(미니 배치를 크게 하면 학습 시간을 단축하는 효과가 있음)
MLPClassifier는 가중치 초기화할 때와 학습 도중에 미니배치를 구성할 때
난수를 사용하기 때문에 시행할 때 마다 다른 결과를 얻게 됨

<https://www.kaggle.com/yukyungchoi/2023-ai-w3-p4/>

4.9 다층 퍼셉트론 프로그래밍

4.9.2 MNIST의 데이터셋으로 확장하기

>> 실습 4-4. 다층 퍼셉트론으로 MNIST 데이터셋 분류하기

```
1 from sklearn.datasets import fetch_openml
2 from sklearn.neural_network import MLPClassifier
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # MNIST 데이터셋을 읽고 훈련 집합과 테스트 집합으로 분할
7 mnist=fetch_openml('mnist_784')
8 mnist.data=mnist.data/255.0
9 x_train=mnist.data[:60000]; x_test=mnist.data[60000:]
10 y_train=np.int16(mnist.target[:60000]); y_test=np.int16(mnist.target[60000:])
11
12 #MLP 분류기 모델을 학습
13 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001)
14 mlp.fit(x_train, y_train)
15
16 # 테스트 집합으로 예측
17 res = mlp.predict(x_test)
18
19 # 혼동 행렬
20 conf = np.zeros((10,10), dtype=np.int16)
21 for i in range(len(res)):
22     conf[res[i]][y_test[i]]+=1
23 print(conf)
24
25 # 정확률 계산
26 no_correct=0
27 for i in range(10):
28     no_correct+=conf[i][i]
29 accuracy=no_correct/len(res)
30 print("테스트 집합에 대한 정확률은", accuracy*100, "%입니다")
```

```
Iteration 56, loss = 0.00330643
Iteration 57, loss = 0.00319682
Iteration 58, loss = 0.00298106
Iteration 59, loss = 0.00324728
Iteration 60, loss = 0.00277761
Iteration 61, loss = 0.00260477
Iteration 62, loss = 0.00239894
Iteration 63, loss = 0.00222857
Iteration 64, loss = 0.00205575
Iteration 65, loss = 0.00198258
Iteration 66, loss = 0.00187042
Iteration 67, loss = 0.00168961
Iteration 68, loss = 0.00161625
Iteration 69, loss = 0.00154800
Iteration 70, loss = 0.00164534
Iteration 71, loss = 0.00212716
Iteration 72, loss = 0.00174860
Iteration 73, loss = 0.00133057
Iteration 74, loss = 0.00124871
Iteration 75, loss = 0.00116872
Iteration 76, loss = 0.00108250
Iteration 77, loss = 0.00104273
Iteration 78, loss = 0.00096834
Iteration 79, loss = 0.00093219
Iteration 80, loss = 0.00088897
Iteration 81, loss = 0.00084531
Iteration 82, loss = 0.00081664
Iteration 83, loss = 0.00080499
Iteration 84, loss = 0.00078998
Training loss did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
[[ 968   0   4   0   1   1   5   1   4   1]
 [   0 1122   2   2   0   0   2   5   1   2]
 [   2   4 1002   8   3   0   4   9   4   0]
 [   1   1   6  985   1  10   1   3   6   5]
 [   1   0   2   0  964   1   1   1   4   8]
 [   1   1   0   5   0  869   5   0   3   3]
 [   2   2   2   0   5   5  940   0   1   0]
 [   1   1   4   2   2   0   0  997   3   5]
 [   4   4   9   4   0   5   0   3  943   1]
 [   0   0   1   4   6   1   0   9   5  984]]
테스트 집합에 대한 정확률은 97.74000000000001 %입니다
```

<https://www.kaggle.com/yukyungchoi/2023-ai-w3-p4/>

4.10 하이퍼 매개변수 최적화

- 하이퍼 매개변수는 모델의 구조와 학습 과정을 제어하는 역할을 함
- 다층 퍼셉트론에서는 은닉 노드의 개수, 학습 규칙에 있는 학습률 등이 하이퍼 매개 변수임
 - 예) MLP를 이용한 MNIST 문제
- [실습4-3]에서 사용한 은닉 노드 100개, 학습률 0.001은 어떻게 설정한 것일까?
 - 임의로 설정한 값을 사용해왔고, 이번 장에서는 좀 더 체계적인 방법을 공부하겠음
- 데이터에 따라 최적의 하이퍼 매개변수 값이 다르므로 하이퍼 매개변수 최적화는 까다로운 문제에 해당됨

4.10 하이퍼 매개변수 최적화

4.10.1 하이퍼 매개변수 살피기

예) MLP를 이용한 MNIST 문제

```
11 #MLP 분류기 모델을 학습
12 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001, batch_size=32, max_iter=300, solver='sgd', verbose=True)
13 mlp.fit(x_train, y_train)
14
```

- MLPClassifier에는 6개의 매개변수가 사용됨
- Verbose=True
 - 학습 도중에 발생하는 정보를 출력하는 방식을 지정
 - True로 설정하면, 세대마다 학습 과정을 상세히 출력하라는 의미
 - False로 설정하면, 정보가 출력되지 않음
- Hidden_layer_sizes=(100)
 - 퍼셉트론의 구조를 제어하는 하이퍼 매개변수
 - 노드가 100개인 은닉층 하나를 두라는 의미
 - Ex) hidden_layer_sizes=(100,80)으로 설정하면 노드가 100개인 은닉층과 80개인 은닉층을 두어 은닉층이 2개인 다층 퍼셉트론이 됨

4.10 하이퍼 매개변수 최적화

4.10.1 하이퍼 매개변수 살펴보기

```
11 #MLP 분류기 모델을 학습
12 mlp = MLPClassifier(hidden_layer_sizes=(100), learning_rate_init=0.001, batch_size=32, max_iter=300, solver='sgd', verbose=True)
13 mlp.fit(x_train, y_train)
14
```

- Learning_rate_init=0.001
 - 학습률을 0.001 로 설정
- Batch_size=32
 - 미니배치 크기를 32로 설정
- Max_iter=300
 - 최대 세대수를 300 으로 설정
- Solver='sgd'
 - 최적화 알고리즘을 스토케스틱 경사 하강법으로 설정

4.10 하이퍼 매개변수 최적화

4.10.1.1 함수의 API

- 함수를 설명하는 공식 문서에 따르면 MLPClassifier 함수의 API는 다음과 같음 (매개변수 총 23개)

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100, ),
activation='relu', solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_
iter=200, shuffle=True, random_state=None, tol=0.0001, verbose=False, warm_
start=False, momentum=0.9, nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_
no_change=10, max_fun=15000)
```

- 명시적으로 설정한 매개변수 6개(파란색으로 표기)를 제외한 나머지 매개변수는 함수 API가 제공하는 기본값을 사용함
 - 예) activation='relu'가 기본값이므로 활성화함수로 ReLU를 사용함
 - 예) shuffle=True 는 새로운 세대를 시작할 때마다 훈련 집합에 있는 샘플의 순서를 뒤섞으라는 뜻임
- 매개변수의 종료 조건은 OR로 동작함
 - max_iter=300으로 설정하여 명령했음에도 불구하고 출력 결과를 살펴보면, 109세대에서 학습을 멈추었음
 - 이는 기본값으로 설정된 인수 tol=0.0001과 n_iter_no_change=10 때문이며, 이 설정은 10세대 동안 손실 함수의 감소량이 0.0001 이하면 멈추라는 의미임

4.10 하이퍼 매개변수 최적화

4.10.1.2 하이퍼 매개변수 설정 가이드라인

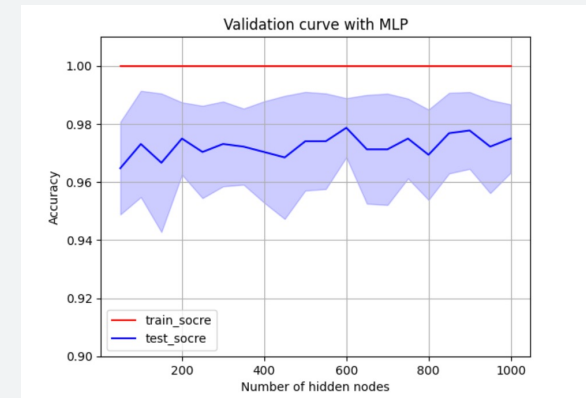
- 기계 학습 모델로 좋은 성능을 얻으려면 <하이퍼 매개변수를 적절하게 설정>해야 하며, 많은 수의 하이퍼 매개변수가 존재하기 때문에 적절한 값의 조합을 알아내는 일은 어려운 일임
- 복잡한 신경망 모델은 하이퍼 매개변수가 훨씬 많아 좋은 값을 설정하는 일이 더욱 어려운 일임
- <하이퍼 매개변수 설정의 어려움을 극복>하는 방법은 경험 규칙 ^{heuristics} 을 따라하는 것
 - (1안) 기계 학습, 딥러닝 관련된 학술대회나 학술지에 발표된 논문은 실험에 사용한 설정을 공개하고 있음
 - (2안) 라이브러리를 제작한 컴퓨터공학자는 하이퍼 매개변수의 기본값을 설정할 때 논문에 제시된 설정과 자신의 경험 등을 종합해 가장 적절하다고 판단되는 값을 사용하고 있음
 - 데이터에 따라 적절한 하이퍼 매개변수 값이 다르지만, 기본값을 사용하면 적절한 범위에 들 가능성이 높음
- <하이퍼 매개변수 설정의 어려움을 극복>하는 또 다른 방법은 최적화 알고리즘을 사용하는 것
 - 경험 규칙이 두루 적용되는 하이퍼 매개변수가 있는 반면 데이터에 민감하여 주어진 데이터에 맞는 최적값을 새로 찾아야 하는 하이퍼 매개변수도 있음
 - 중요하다고 판단되는 하이퍼 매개변수를 골라 하이퍼 매개변수 최적화 알고리즘을 적용하는 것
 - 3~4개를 골라 다중 하이퍼 매개변수 최적화 알고리즘을 적용할 때는 GridSearchCV 함수를 사용

4.10 하이퍼 매개변수 최적화

4.10.2 단일 하이퍼 매개변수 최적화: validation_curve 함수 이용

- [프로그램 4-5]는 은닉층의 노드 개수라는 하이퍼 매개변수를 최적화

```
12 # 다층 퍼셉트론을 교차 검증으로 성능 평가 (소요 시간 측정 포함)
13 start=time.time()
14 mlp=MLPClassifier(learning_rate_init=0.001, batch_size=32, max_iter=300, solver='sgd')
15 prange=range(50,1001,50)
16
17 #
18 train_score, test_score=validation_curve(mlp, x_train, y_train, param_name='hidden_layer_sizes', param_range=prange, cv=10, scoring='accuracy', n_jobs=4)
19 end=time.time()
20 print("하이퍼 매개변수 최적화에 걸린 시간은", end-start, "초입니다.")
21
22 # 교차 검증 결과의 평균과 분산 구하기
23
24 train_mean = np.mean(train_score, axis=1)
25 train_std = np.std(train_score, axis=1)
26 test_mean = np.mean(test_score, axis=1)
27 test_std = np.std(test_score, axis=1)
28
29 # 성능 그래프 그리기
30 plt.plot(prange, train_mean, label="train_socre", color='r')
31 plt.plot(prange, test_mean, label="test_socre", color='b')
32 plt.fill_between(prange, train_mean-train_std, train_mean+train_std, alpha=0.2, color='r')
33 plt.fill_between(prange, test_mean-test_std, test_mean+test_std, alpha=0.2, color='b')
34 plt.legend(loc='best')
35 plt.title("Validation curve with MLP")
36 plt.xlabel("Number of hidden nodes")
37 plt.ylabel("Accuracy")
38 plt.ylim(0.9, 1.01)
39 plt.grid(axis="both")
40 plt.show()
41
42 best_number_nodes=prange[np.argmax(test_mean)] # 최적의 은닉 노드 개수
43 print("\n최적의 은닉층의 노드 개수는", best_number_nodes, "개입니다 \n")
```



은닉층 600개 일 때 가장 성능 우수

<https://www.kaggle.com/yukyungchoi/2023-ai-w3-p5/>

4.11 실습

- [연습 문제] 캐글 리더보드에 MNIST 예측 값 제출하기
- 캐글 리더보드 주소
 - Private Link : <https://www.kaggle.com/t/9fd8bc8f753b4cbab7b70eefd8a60bb8>
 - 반드시 Private Link로 접근할 것
- 캐글 리더보드에 답안 제출법
 - 깃허브 PDF 참고
- Numpy, Pandas 사용법
 - <https://www.youtube.com/watch?v=xEXzE0oWFvw>
 - <https://www.youtube.com/watch?v=sFJyzPD4rw0>

4.12 과제

- [문제] 과제로 제시된 리더보드(과제1~4)의 문제를 해결하라
 - 모델 : SVM, SLP, MLP
 - (힌트) 4.11에서 배운 MNIST 베이스라인 코드 참고
- 제출 방식 : 이메일로 보고서 제출
 - 이메일 주소 : admin@rcv.sejong.ac.kr
 - 이메일 제목 : [2025-2] [인공지능] 3주차 과제 제출 (학번_이름)
 - 보고서 분량 제한 없음
 - 캐글 노트북은 안내된 방식에 따라 공유

4.12 과제

- [과제1 리더보드] IRIS 데이터 분류하기 (SVM, MLP, SLP)

<https://www.kaggle.com/t/543455411eec47c7ae13f5e50d27bf72>

- [과제2 리더보드] DIGITS 손글씨 분류하기 (SVM, MLP, SLP)

<https://www.kaggle.com/t/ee430a9aebb04834853c6453f9425602>

- [과제3리더보드] 20NewsGroups 문서 분류하기 (SVM, MLP, SLP)

<https://www.kaggle.com/t/fc6fac2be45049f1a808c7c5b5516266>

- [과제4 리더보드] 유명인 얼굴 분류하기 (SVM, MLP, SLP)

<https://www.kaggle.com/t/3458558e4ec741eea27f1091b4947bf7>